UNIT-II

In-Memory Data Middleware

Need for In-Memory Data Middleware, Role of Caching in Distributed Systems, Introduction to Redis, Redis Architecture, Key features, Single-Threaded Execution Model, Redis Data Structures: Strings, Lists, Sets, Sorted Sets, Hashes, Using Redis as a Cache and Session Store, Redis Pub/Sub Messaging Model, Persistence Mechanisms: RDB and AOF, Replication and High Availability, Redis Clustering, Comparison of Redis with Traditional Relational Databases.

In-Memory Data Middleware

The modern digital landscape is characterized by an unprecedented demand for speed, scalability, and real-time responsiveness. Applications ranging from high-frequency trading platforms to global e-commerce systems and sophisticated machine learning pipelines require data access and processing at microsecond speeds. This critical requirement has exposed a fundamental limitation in traditional data architectures, leading to the emergence of In-Memory Data Middleware (IMDM) as an essential component of high-performance distributed systems.

In-memory data middleware is a class of technologies designed to sit between applications and traditional persistent data storage systems, holding data in Random Access Memory (RAM) as the primary medium for data storage and computation. It is an architectural pattern designed to overcome the slower mechanical disks or even solid-state drives (SSDs), providing a substantial boost to application performance. The two primary types of in-memory data middleware are in-memory databases (IMDBs) and in-memory data grids (IMDGs).

In-Memory Databases vs. In-Memory Data Grids

While both technologies leverage in-memory storage, they have key differences:

Feature	**In-Memory Database (IMDB)**	**In-Memory Data Grid (IMDG)**
Primary Function	A full-featured, standalone database that runs in memory.	A distributed data store that often sits between applications and a persistent database.
Scalability	Can have limitations in distributed environments, especially with complex queries like SQL joins.	Designed for massive horizontal scalability using a Massively Parallel Processing (MPP) architecture.
Data Processing	Typically processes data on the server side, with the application as a client.	Can co-locate business logic with the data, reducing network latency.

The Need for In-Memory Data Middleware

The demand for in-memory data middleware stems from the limitations of traditional disk-based databases in meeting the real-time performance requirements of modern applications. Here are the key drivers for its adoption:

- **Low Latency and High Throughput:** The primary advantage is a dramatic reduction in data access times. By storing data in RAM, in-memory solutions can deliver microsecond read latencies and single-digit millisecond write latencies. This is crucial for applications that require real-time responses, such as financial trading platforms, real-time analytics, and online gaming.
- **Enhanced Scalability:** Modern architectures, particularly those based on microservices, demand linear scalability. Traditional databases often struggle to scale horizontally without significant complexity or cost. IMDM, through techniques like Data Partitioning (Sharding), allows data to be distributed across numerous nodes. This means that as application load increases, new nodes can be added to the IMDM cluster to increase both capacity and processing power linearly. This capability is crucial for maintaining performance in elastic, cloud-native environments.
- **Data-Intensive AI/ML Workloads:** Artificial Intelligence and Machine Learning models are data-hungry, requiring high-bandwidth access to massive datasets for training and inference. The speed of data retrieval can become a significant data bottleneck in deep learning stacks. In-memory data middleware provides the speed needed for these workloads, such as real-time feature stores for machine learning models and ultra-fast vector searches for Retrieval-Augmented Generation (RAG) pipelines.
- **Improved User Experience:** For web and mobile applications, in-memory data middleware can be used for session management, ensuring a smooth and responsive user experience even during traffic spikes. By keeping session data in memory, applications can instantly retrieve and update user information.
- **Reduced Database Load:** By acting as a caching layer, in-memory data middleware can absorb a significant portion of the read and write operations, reducing the load on backend databases. This can extend the life of existing database infrastructure and prevent performance bottlenecks.

Real-World Applications

The need for In-Memory Data Middleware is most pronounced in industries where speed and concurrency are paramount.

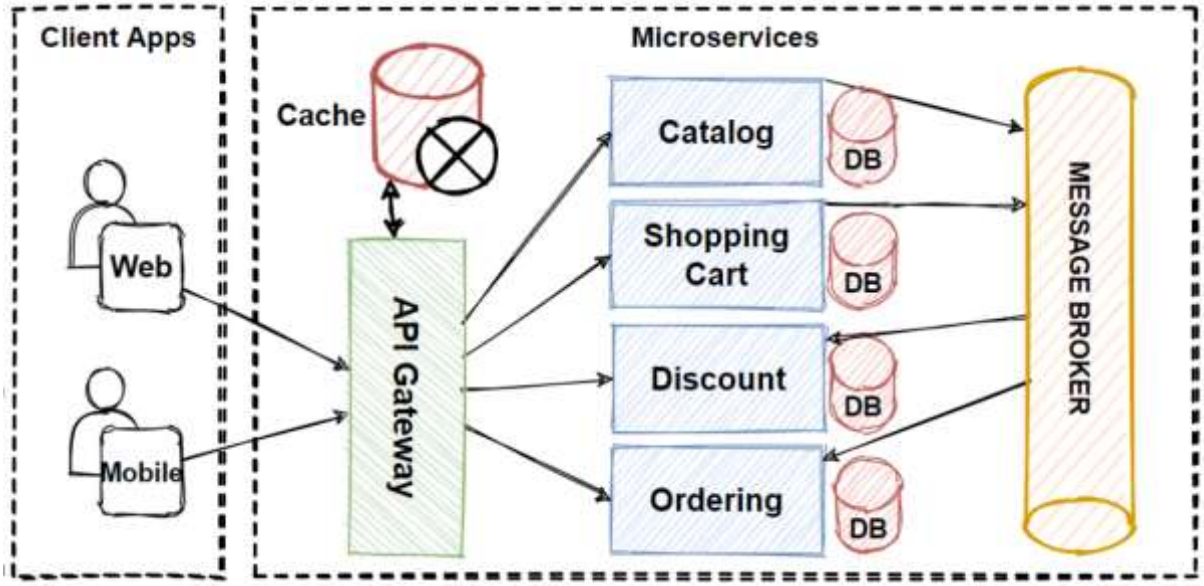
Industry	Use Case	Necessity
Financial Services	High-Frequency Trading, Fraud Detection, Risk Management.	Sub-millisecond latency is required to execute trades and detect anomalies in real-time.
E-commerce & Retail	Shopping Cart Management, Session State, Real-Time Inventory.	Must handle millions of concurrent user sessions and provide instant inventory updates during peak sales.
Telecommunications	Subscriber Data Management, Real-Time Billing, Network Monitoring.	Requires instant access to subscriber profiles and real-time processing of massive call detail records (CDRs).
Gaming	Real-Time Leaderboards, Player State Synchronization.	Low-latency data synchronization is essential for a seamless and fair multiplayer experience.
Logistics & IoT	Real-Time Asset Tracking, Sensor Data Aggregation.	Needs to ingest and process high-velocity data streams from thousands of devices for immediate decision-making.

Role of Caching in Distributed Systems

Architectural Visualization

The following figures illustrate how distributed caching integrates into a modern system architecture and the flow of data for different strategies.

Figure 1: Distributed Cache in a Microservices Architecture



This diagram shows how a central cache layer can be shared across multiple microservices (Catalog, Shopping Cart, Discount, Ordering), all accessed through an API Gateway. This centralized, distributed cache ensures that data consistency is maintained across services and provides a single source of truth for cached data.

Caching is a fundamental technique in computer science, and its role becomes paramount in the context of **distributed systems**. In essence, a cache is a high-speed data storage layer that stores a subset of data, typically transient in nature, so that future requests for that data can be served faster than retrieving it from the primary, slower data store.

In a distributed system, the cache itself is often distributed across multiple servers or nodes, forming a **Distributed Cache**. This architecture allows the cache to scale horizontally, handling massive volumes of requests and data without becoming a single point of failure or a performance bottleneck.

Primary Roles and Benefits of Distributed Caching

The integration of a distributed cache layer into a system architecture provides several critical advantages that address the inherent challenges of scale and latency in distributed environments.

1. Enhanced Performance and Reduced Latency

The most significant role of caching is to dramatically improve the response time of applications. Data access from a cache (which resides in fast memory, or RAM) is orders of magnitude faster than accessing data from a persistent store like a database (which involves disk I/O and network latency). By serving frequently accessed data directly from the cache, the system achieves microsecond-level latency, which is essential for real-time applications and a superior user experience [1].

2. Increased Scalability and Throughput

Distributed caching allows the system to handle a much higher volume of requests. By offloading read traffic from the primary database, the cache acts as a buffer, preventing the database from being overwhelmed during peak loads. Since the cache is distributed across multiple nodes, it can scale horizontally to accommodate growing data size and request throughput, a capability often limited in monolithic databases.

3. Reduced Load on Backend Services

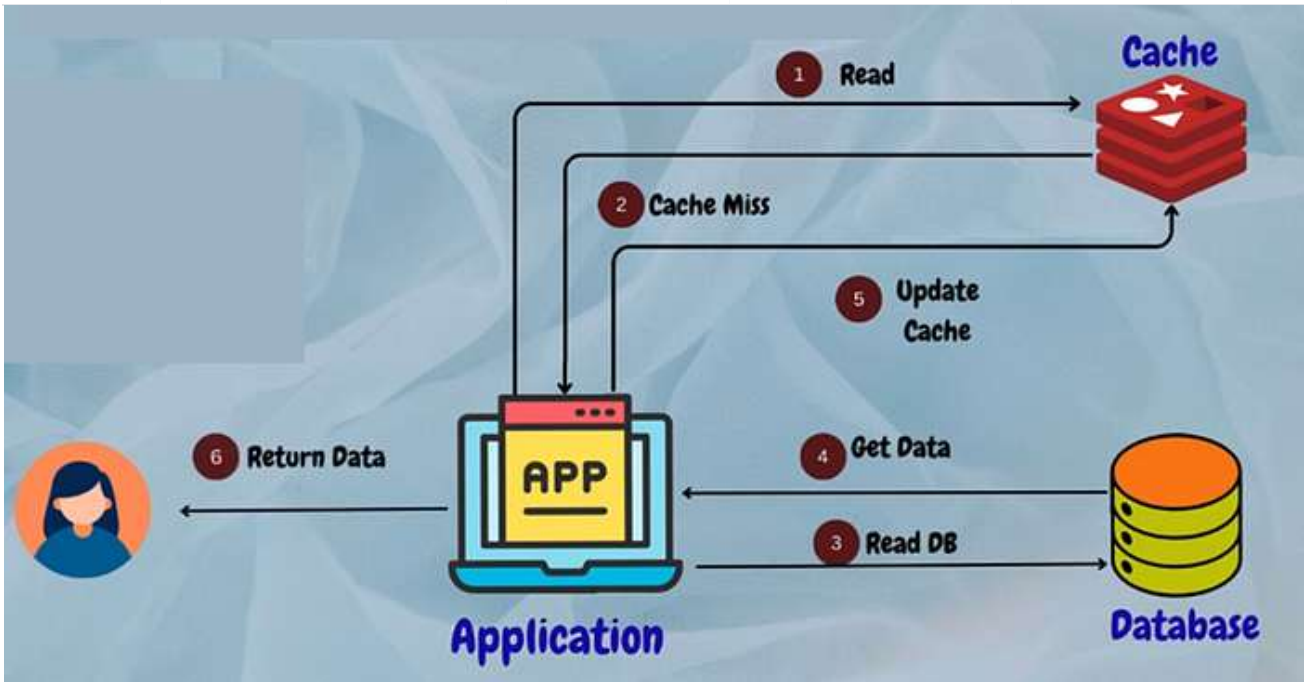
A cache acts as a protective layer for the backend database and other persistent services. A high cache hit ratio means fewer requests reach the database, reducing its processing load, I/O operations, and network traffic. This not only improves the database's performance but also reduces operational costs and extends the lifespan of the underlying infrastructure.

Distributed Caching Strategies

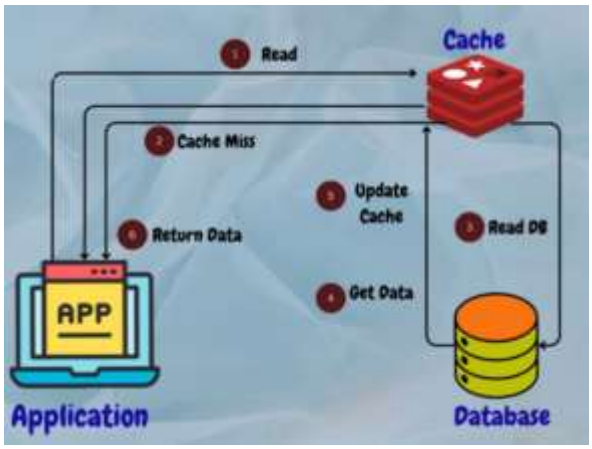
The way an application interacts with the cache and the primary data store is defined by its caching strategy. Choosing the correct strategy is crucial for optimizing performance, maintaining data consistency, and simplifying application logic.

Strategy	Description	Read Flow	Write Flow	Example
Cache-Aside (Lazy Loading)	The application is responsible for managing the cache. It checks the cache first; on a miss, it fetches from the database, updates the cache, and returns the data.	Application checks cache; on miss, reads from DB, writes to cache.	Application writes directly to DB, then optionally invalidates or updates the cache.	MongoDB caching frequently accessed user profiles in a social media platform
Read-Through	The cache is responsible for fetching data from the database on a cache miss. The application only interacts with the cache.	Application requests data from cache; on miss, cache fetches from DB and returns data.	Application writes directly to cache.	Memcached used for caching frequently accessed database queries in a web application.
Write-Through	Data is written synchronously to both the cache and the database. This ensures data consistency at the cost of increased write latency.	Same as Read-Through or Cache-Aside.	Application writes to cache, which synchronously writes to the database.	Redis cache with write-through caching for user session management.
Write-Back (Write-Behind)	Data is written to the cache first, and the write to the database is performed asynchronously later, often in batches.	Same as Read-Through or Cache-Aside.	Application writes to cache, which acknowledges immediately. Cache asynchronously writes to the database.	A database system caching write operations in memory before committing them to disk to improve overall system responsiveness and throughput.

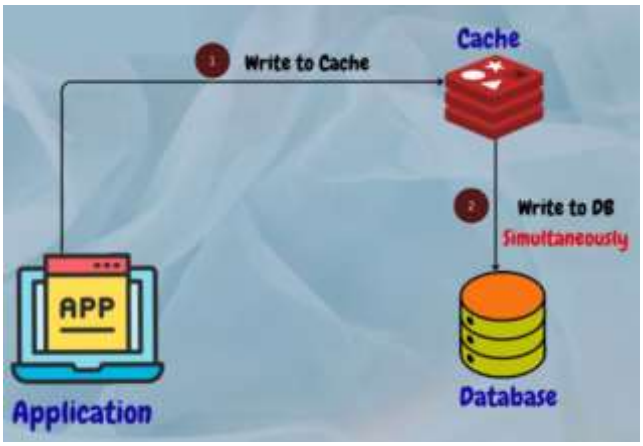
Strategy	Description	Read Flow	Write Flow	Example
Write-Around	Data is written directly to the database, bypassing the cache entirely. Only subsequent reads populate the cache.	Same as Cache-Aside.	Application writes directly to DB, bypassing the cache.	A file storage system where large files are written directly to disk without being cached to optimize cache utilization for frequently accessed files.



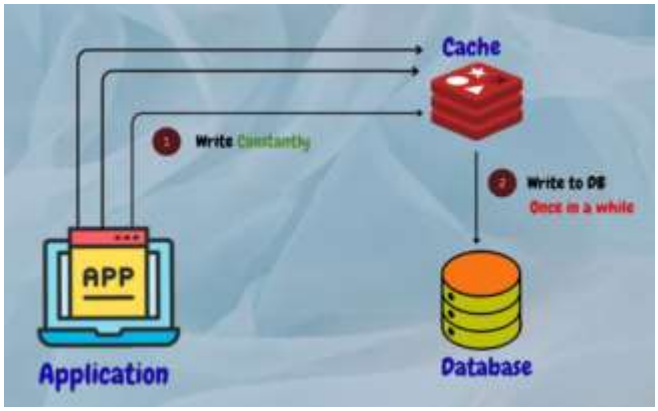
Cache-Aside (Lazy loading)

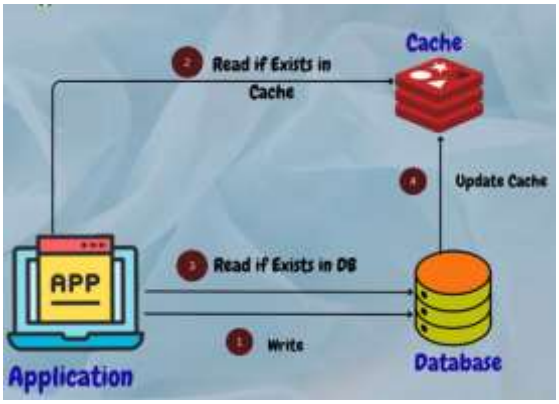


Read-Through Cache



Write-Through Cache





Write-Back Cache **Write-Around Cache**

Cache Invalidation and Eviction

A critical challenge in distributed caching is maintaining **data consistency**—ensuring the data in the cache accurately reflects the data in the primary store. This is managed through **invalidation** and **eviction** policies.

Cache Invalidation

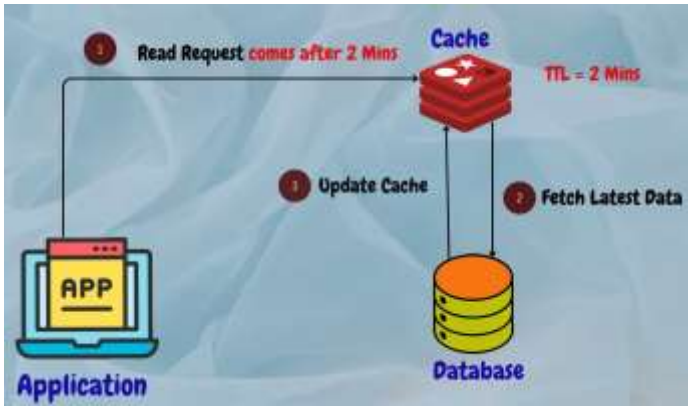
Cache invalidation methods confirm that the cached data matches the latest information from the original source. Cache invalidation is the process of removing or updating cached data when it becomes stale or outdated. This ensures that the cached data remains accurate and reflects the most recent changes from the source of truth, such as a database or a web service.

“Purge,” “refresh,” and “ban” are commonly used cache invalidation methods that are frequently used in the application cache, content delivery networks (CDNs), and web proxies.

Cache Invalidation Methods:

1. **Time-Based Invalidation**: Cached data is invalidated after a specified period to ensure freshness. Example: Expire user authentication tokens in the cache after a certain time interval.

Time-to-Live (TTL) Expiration: This method involves setting a time-to-live value for cached content, after which the content is considered stale and must be refreshed. When a request is received for the content, the cache checks the time-to-live value and serves the cached content only if the value hasn’t expired. If the value has expired, the cache fetches the latest version of the content from the origin server and caches it.



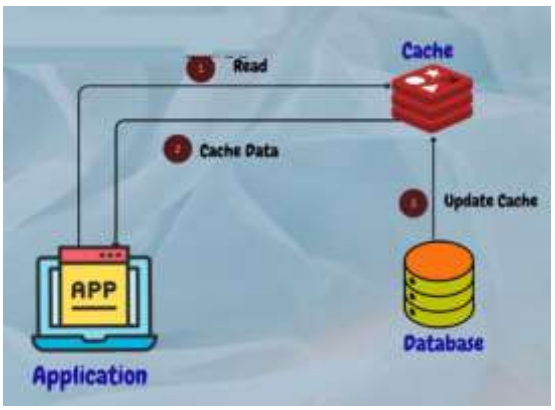
2. Event-Based Invalidation: The database or application explicitly notifies the cache to invalidate a specific entry upon a data change (e.g., a database trigger or a message queue event).

Purge: The purge method removes cached content for a specific object, URL, or a set of URLs. It's typically used when there is an update or change to the content and the cached version is no longer valid. When a purge request is received, the cached content is immediately removed, and the next request for the content will be served directly from the origin server.

Refresh: Fetches requested content from the origin server, even if cached content is available. When a refresh request is received, the cached content is updated with the latest version from the origin server, ensuring that the content is up-to-date. Unlike a purge, a refresh request doesn't remove the existing cached content; instead, it updates it with the latest version.

Ban: The ban method invalidates cached content based on specific criteria, such as a URL pattern or header. When a ban request is received, any cached content that matches the specified criteria is immediately removed, and subsequent requests for the content will be served directly from the origin server.

Stale-While-Revalidate: This method is used in web browsers and CDNs to serve stale content from the cache while the content is being updated in the background. When a request is received for a piece of content, the cached version is immediately served to the user, and an asynchronous request is made to the origin server to fetch the latest version of the content. Once the latest version is available, the cached version is updated.



This method ensures that the user is always served content quickly, even if the cached version is slightly outdated.

Cache Eviction

Eviction is necessary when the cache reaches its capacity limit. Policies determine which entries to remove to make space for new data.

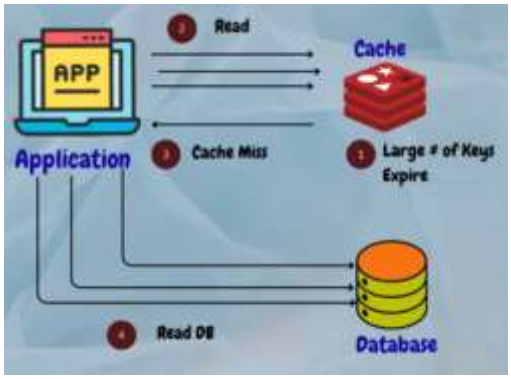
There are various cache eviction policies:

1. **Least Recently Used (LRU):** Removes the least recently accessed items when space is needed for new entries. For instance, in a web browser's cache using LRU, if the cache is full and a user visits a new webpage, the least recently accessed page is removed to make room for the new one.
2. **Most Recently Used (MRU):** Evicts the most recently accessed items to free up space when necessary. For example, in a mobile app cache, if the cache is full and a user accesses a new feature, the most recently accessed feature is evicted to accommodate the new data.
3. **First-In-First-Out (FIFO):** Evicts items based on the order they were added to the cache. In a FIFO cache, the oldest items are removed first when space is required. For instance, in a messaging app, if the cache is full and a new message arrives, the oldest message in the cache is evicted.
4. **Least Frequently Used (LFU):** Removes items that are accessed the least frequently. LFU keeps track of access frequency, and when space is limited, it removes items with the lowest access count. For example, in a search engine cache, less frequently searched queries may be evicted to make room for popular ones.
5. **Random Replacement (RR):** Selects items for eviction randomly without considering access patterns. In a cache using RR, any item in the cache may be removed when space is needed. For instance, in a gaming app cache, when new game assets need to be cached, RR randomly selects existing assets to replace.

Each cache eviction policy has its advantages and disadvantages, and the choice depends on factors such as the application's access patterns, memory constraints, and performance requirements.

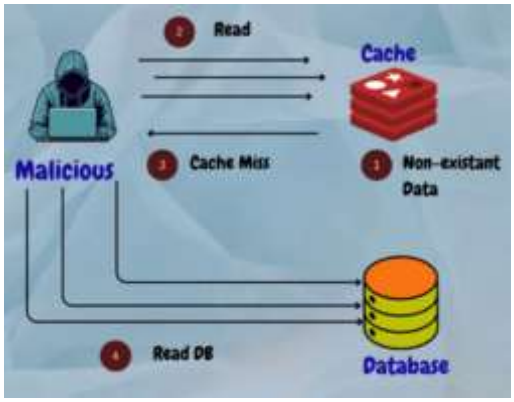
Common Cache-Related Problems and Solutions:

1. Thundering Herd Problem: Occurs when multiple requests simultaneously trigger cache misses, overwhelming the system.



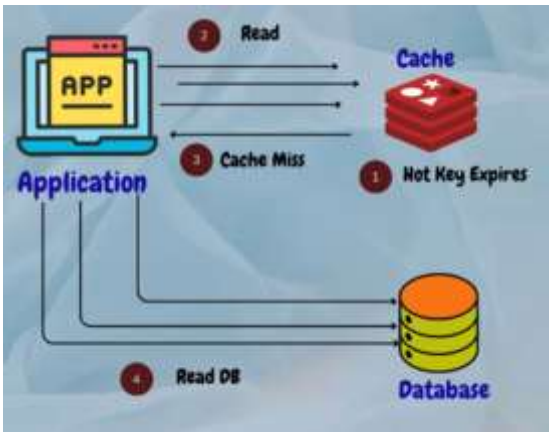
Solution: Implement cache warming techniques to pre-load frequently accessed data into the cache during low-traffic periods.

2. Cache Penetration: Happens when malicious users bombard the system with requests for non-existent data, bypassing the cache and causing unnecessary load on the backend.



Solution: Implement input validation and rate limiting to mitigate the impact of cache penetration attacks.

3. Cache Breakdown: Occurs when the cache becomes unavailable or unresponsive, leading to increased load on the backend database.



Solution: Implement cache redundancy and failover mechanisms to ensure high availability and fault tolerance.

4. Cache Crash: Happens when the cache experiences failures or crashes due to software bugs or hardware issues. Solution: Regularly monitor cache health and performance, and implement automated recovery mechanisms to quickly restore cache functionality.

In companies like Amazon, Netflix, Meta, Google and eBay distributed caching solutions like **Redis or Memcached** are commonly used for caching dynamic data, while CDNs are employed for caching static content and optimizing content delivery to users.

Introduction to Redis

Redis stands for **RE**mote **DI**ctionary **S**erver. It was written in C by Salvatore Sanfilippo in 2006. **Redis** is an in-memory data store that can behave as a database, caching system, or even a **message broker**. Due to its in-memory nature, the Redis server will always be running on the RAM and all the data will actually be stored on the RAM itself. This comes with the great advantage that accessing the data from Redis can be much faster, but there's a limitation too. If the Redis server is restarted, all of our data is lost. To prevent this, Redis also supports storing the data at regular intervals to persistent storage so that data loss never happens.

It is a NoSQL advanced *key-value* data store. The read and write operations are very fast in Redis because it stores all data in memory. The data can also be stored on the disk and can be written back to the memory. It is often referred to as a data structure server because keys can contain strings, hashes, lists, sets, sorted sets, bitmaps, and hyperloglogs.

NoSQL means **Not only SQL**. The NoSQL databases don't define database structures like tables, nor do they support queries, such as SQL SELECT. The data is mostly stored as JSON objects or key-value pairs.

Since Redis stores its data in memory, it is most commonly used as a cache. Some of the organizations that are using Redis are Twitter, GitHub, Instagram, Pinterest, and Snapchat.

Using Redis as a caching system

Redis can be used as a caching system to speed up the performance of web applications. An example that we already discussed is the DNS looking up the IP address for a given URL. When Redis is used as a cache, it stores frequently accessed data in memory, which reduces the number of database calls required to retrieve the data. This can significantly improve the performance of web applications by reducing the latency caused by database calls. Redis also allows for the setting of expiration times for cached data, which means that old data can be automatically removed from the cache to make room for new data. Some of the scenarios where Redis is a great choice as a caching system include:

- Applications that require low-latency responses
- Applications that frequently access the same data
- Applications that need to reduce the load on their database

A real-world example is Twitter. It's one of the most popular social media platforms in the world, with millions of active users tweeting, retweeting, and liking posts every second. To handle such a massive volume of data, Twitter uses a combination of multiple technologies, including Redis as a caching system.

Twitter uses Redis as an in-memory data store to cache frequently accessed data, such as user profiles, tweets, timelines, and trends. By caching data in Redis, Twitter can reduce the number of requests sent to its databases, which helps to improve the performance and scalability of the platform.

Here are some specific ways in which Twitter uses Redis as a caching system:

- **User profiles:** Twitter caches user profiles in Redis to reduce the number of database calls required to retrieve user information. Whenever a user logs in to Twitter, their profile information is loaded from the Redis cache, which helps to improve the response time of the platform.
- **Tweets:** Twitter caches tweets in Redis to speed up the process of retrieving tweets for users. When a user visits their timeline, Twitter fetches the most recent tweets from the Redis cache, which can significantly reduce the latency of the platform.

- **Timelines:** Twitter caches timelines in Redis to reduce the number of database calls required to fetch tweets for a particular user's timeline. When a user requests their timeline, Twitter loads the most recent tweets from the Redis cache, which helps to improve the performance of the platform.
- **Trends:** Twitter caches trending topics in Redis to reduce the number of database calls required to fetch the most popular topics. Whenever a user visits the trending topics section of the platform, Twitter retrieves the data from the Redis cache, which can significantly reduce the response time of the platform.

Using Redis as a message broker system

Redis can be used as a message broker to facilitate communication between different components of a distributed system. Redis supports a publish-subscribe messaging model, where publishers send messages to a channel, and subscribers listen to those channels for incoming messages. This allows different components of a distributed system to communicate with each other in a decoupled way. Redis also supports various other messaging patterns, such as request-response, where a client sends a message to a server and waits for a response. Some of the scenarios where Redis is a great choice as a message broker include the following:

- Applications that require decoupled communication between components
- Applications that need to handle high volumes of messages
- Applications that need to perform messaging operations quickly and efficiently

A real-world example is Uber. It uses Redis as a message broker to facilitate communication between various microservices in its distributed system. **Microservices** are individual components of a larger system that communicate with each other through APIs or messaging systems like Redis.

In Uber's case, Redis is used to implement a publish-subscribe messaging pattern, where publishers send messages to a channel, and subscribers listen to those channels for incoming messages. Uber uses this pattern to enable communication between various components of its system, including trip management, real-time dispatch, and payment processing.

- **Book a ride:** When a passenger requests a ride in the Uber app, a message is published to a Redis channel indicating the passenger's location, destination, and other ride details. The dispatch service, which is responsible for finding available drivers and assigning them to the ride, subscribes to this channel and receives the message. The dispatch service then sends a message back to Redis indicating which driver has been assigned to the ride.
- **Payment processing system:** When a ride is completed, a message is published to a Redis channel indicating that the ride has ended and the fare amount. The payment processing service subscribes to this channel and receives the message, then processes the payment for the ride.

Using Redis as a message broker provides several benefits for Uber's distributed system. Firstly, it allows for decoupled communication between different microservices, which helps to increase flexibility and scalability. Secondly, Redis supports high-throughput messaging, which allows Uber to handle the high volumes of messages generated by its system. Finally, Redis provides built-in features for handling message retries and failures, which helps to ensure the reliability and consistency of Uber's system.

Redis Architecture

There are several Redis architectures, depending on the use case and scale:

1. Single Redis Instance
2. Redis HA
3. Redis Sentinel
4. Redis Cluster

Depending on your use case and scale, you can decide to use one setup or another.

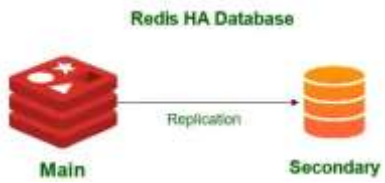
1. **Single Redis Instance**

Single Redis instance is the most straightforward deployment of Redis. It allows users to set up and run small instances that can help them grow and speed up their services. However, this deployment isn't without shortcomings. For example, if this instance fails or is unavailable, all client calls to Redis will fail and therefore degrade the system's overall performance and speed. Given enough memory and server resources, this instance can be powerful. A scenario primarily used for caching could result in a significant performance boost with minimal setup. Given enough system resources, you could deploy this Redis service on the same box the application is running.

Understanding a few Redis concepts on managing data within the system is essential. Commands sent to Redis are first processed in memory. Then, if persistence is set up on these instances, there is a forked process on some interval that facilitates data persistence RDB (very compact point-in-time representation of Redis data) snapshots or AOF (append-only files).

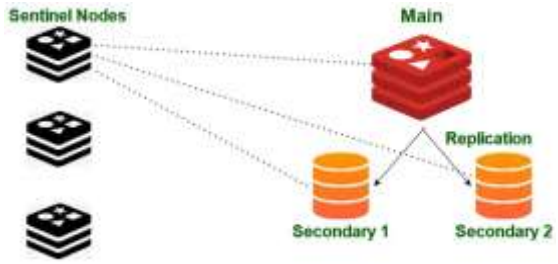
These two flows allow Redis to have long-term storage, support various replication strategies, and enable more complicated topologies. If Redis isn't set up to persist data, data is lost in case of a restart or failover. If the persistence is enabled on a restart, it loads all of the data in the RDB snapshot or AOF back into memory, and then the instance can support new client requests.

2. **Redis HA**



Another popular setup with Redis is the main deployment with a secondary deployment that is kept in sync with replication. As data is written to the main instance it sends copies of those commands, to a replica client output buffer for secondary instances which facilitates replication. The secondary instances can be one or more instances in your deployment. These instances can help scale reads from Redis or provide failover in case the main is lost.

3. **Redis Sentinel**



Sentinel corresponds to a distributed system. It is designed in a way where there is a cluster of sentinel processes working together for coordination of state to provide constant availability of the Redis system. Here are the responsibilities of the sentinel:

- **Monitoring:** Ensuring main and secondary instances are working as expected.
- **Notification:** Notify all the system admins about the events occurring in Redis instances.
- **Management during failure:** Sentinel nodes can start a process during failure if the primary instance is not available for long enough, and enough nodes agree that it is true.

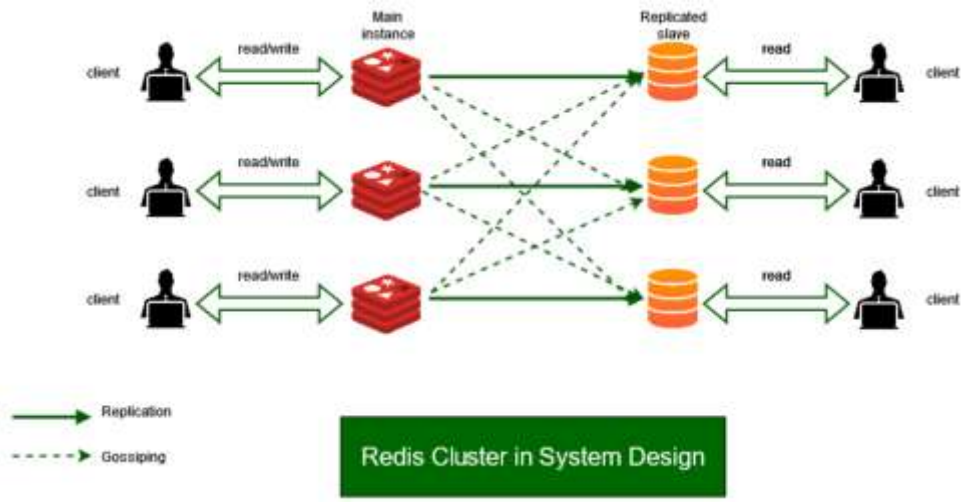
4. **Redis Cluster**

In Redis cluster, we decide to spread the data we are storing across multiple machines, which is known as **Sharding**. So each such Redis instance in the cluster is considered a shard of the whole data.

The Redis Cluster uses **algorithmic sharding**. To find the shard for a given key, we hash the key and mod the total result by the number of shards. Then, using a **deterministic hash function**, meaning that a given key will always map to the same shard, we can reason about where a particular key will be when we read it in the future.

To handle further addition of shards into the system (**resharding**), the Redis cluster uses **Hashslot,** to which all of the data is mapped. Thus, when we add new shards, we simply move

hashslots from shard to shard and simplify the process of adding new primary instances into the cluster.



And to the advantage, this is possible without any downtime, and minimal performance hit. Let's look at an example below:

*Consider the number of hashslots to be **10K**.*
*Instance1 contains hashslots from **0 to 5000**.*
*Instance2 contains hashslots from **5001 to 10000**.*

Now, let's say we need to add another instance, now the distribution of hashslots comes to,

*Instance1 contains hashslots from **0 to 3333**.*
*Instance2 contains hashslots from **3334 to 6667**.*
*Instance3 contains hashslots from **6668 to 10000**.*

What is gossiping in the Redis cluster?

To determine the entire cluster's health, the redis cluster uses **gossiping**. In the example below, we have 3 main instances and 3 secondary nodes of them. All these nodes constantly determine which nodes are currently available to serve requests. Suppose, if enough shards agree that instance1 is not responsive, they can promote instance1's secondary into a primary to keep the cluster healthy. As a general rule of thumb, it is essential to have an odd number of primary nodes and two replicas each for the most robust and fault-tolerant network.

Key Features of Redis:

- **In-Memory Data Storage:** Redis stores the entire dataset in memory, which allows for incredibly fast read and write operations. This makes it ideal for applications that require real-time data processing and low latency.
- **Flexible Data Structures:** Redis is not just a simple key-value store. It supports a variety of data structures, including:
    - **Strings:** For storing text or binary data up to 512MB.

    - **Lists:** Ordered collections of strings, often used for implementing queues.
    - **Sets:** Unordered collections of unique strings.
    - **Hashes:** To store fields and values, similar to objects.
    - **Sorted Sets:** Similar to sets, but each member has an associated score, allowing for ordered retrieval.
    - **Bitmaps and HyperLogLogs:** For space-efficient counting of unique items.
    - **Streams:** A log data structure for managing high-velocity data streams.
- **High Availability and Scalability:** Redis offers features like replication and clustering to ensure high availability and scalability. Replication creates copies of your data, so if one server fails, another can take over. Clustering automatically splits your data across multiple nodes to improve uptime and performance.
- **Persistence:** While Redis is an in-memory database, it also provides options for data persistence. This means your data can be saved to disk and reloaded if the server restarts.
- **Pub/Sub Messaging:** Redis includes a publish/subscribe messaging system, which allows for real-time communication between different parts of an application.
- **Lua Scripting:** You can execute complex operations atomically on the server-side using Lua scripting.
- **Security:** Redis provides several security features, including password authentication, TLS encryption for data in transit, and Access Control Lists (ACLs) for fine-grained user permissions.
- **Extensibility with Redis Modules:** Redis can be extended with modules to add new data types and functionalities. For example, RedisSearch adds full-text search capabilities.

Single-Threaded Execution Model

At its heart, Redis uses a single main thread to execute all client commands. This means that it processes one command at a time in a sequential order. When a client sends a command (like SET, GET, or LPUSH), it goes into a queue. The Redis event loop picks up one command from the queue, executes it, and then picks up the next one.

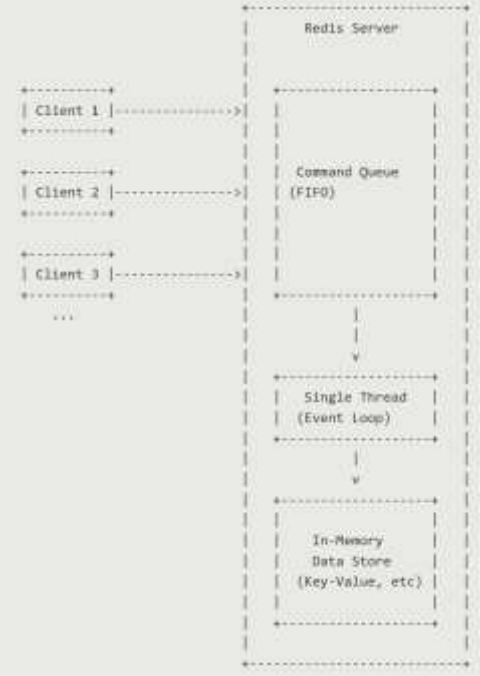
You might think a single thread would be slow, but Redis is incredibly fast for two main reasons:

1. **In-Memory Operations:** Most Redis operations are on data stored in RAM, which is extremely fast. There's no time wasted on disk I/O for the primary operations.
2. **Non-Blocking I/O:** For network communication (like accepting client connections and sending back responses), Redis uses a technique called **I/O multiplexing**. This allows the single thread to efficiently manage multiple client connections at once. Instead of waiting for a slow client to send a request or receive a response, Redis can handle other active clients in the meantime.

This model avoids the complexities of multithreading, such as race conditions and the overhead of locking, which makes the Redis source code simpler and more predictable.
It's important to note that while the *command execution* is single-threaded, Redis does use background threads for less critical tasks like data persistence (saving data to disk) and deleting objects asynchronously.

How it works, step-by-step:

1. **Connections:** Multiple clients connect to the Redis server. Redis uses I/O multiplexing to handle these connections efficiently without blocking.
2. **Commands Queued:** As clients send commands, they are placed into a First-In, First-Out (FIFO) queue.
3. **Sequential Execution:** The single-threaded event loop picks up one command from the queue at a time.
4. **Execution:** The command is executed against the in-memory data store. Since this is in RAM, it's extremely fast.
5. **Response:** A response is sent back to the client, and the loop moves to the next command in the queue.

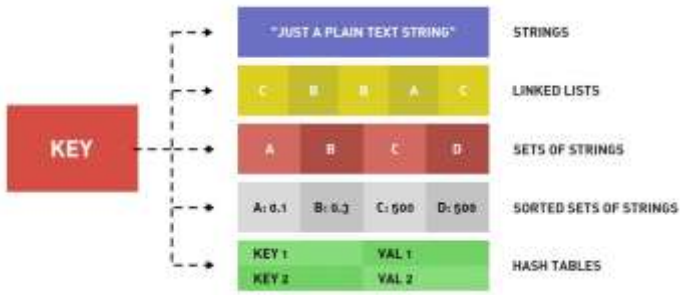


Because each command is atomic and runs sequentially, you never have to worry about two commands modifying the same data at the exact same time.

Redis Data Structures: Strings, Lists, Sets, Sorted Sets, Hashes,

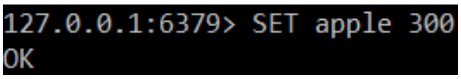
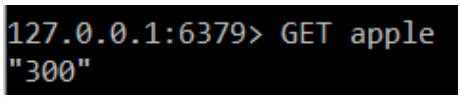
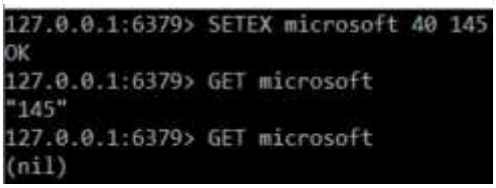
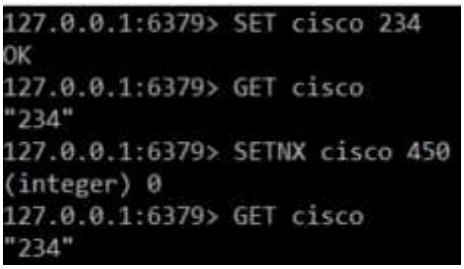
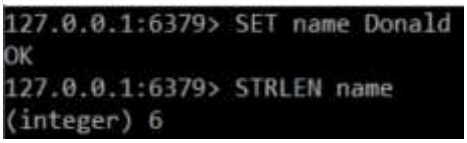
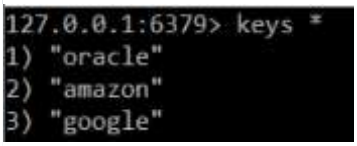
Data Structure	Description	Common Use Cases
Strings	The most basic type, holding a sequence of bytes (text or binary data).	Caching HTML fragments, storing session tokens, simple counters.
Lists	A collection of ordered strings, implemented as a linked list. Allows for fast insertion and deletion at the head or tail.	Implementing message queues, social media timelines, and history lists.
Sets	An unordered collection of unique strings. Supports set operations like union, intersection, and difference.	Storing unique visitors, tracking tags, and implementing access control lists.
Sorted Sets (ZSETs)	Similar to Sets, but each member is associated with a floating-point score, allowing for ordered retrieval by score or lexicographically.	Creating real-time leaderboards, ranking items, and rate limiting.
Hashes	A map between string fields and string values, perfect for representing objects.	Storing user profiles (e.g., user:100 with fields like name, email, status).

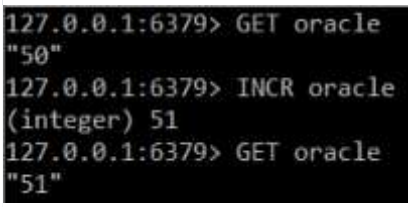
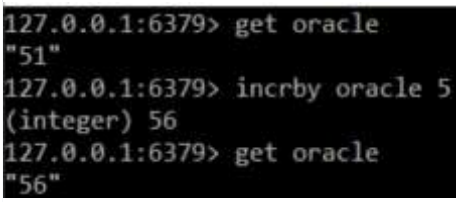
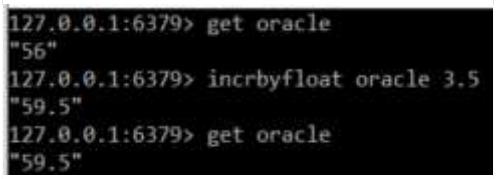
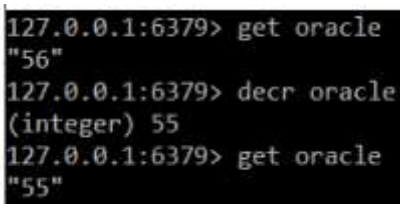
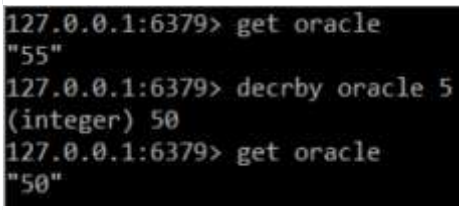
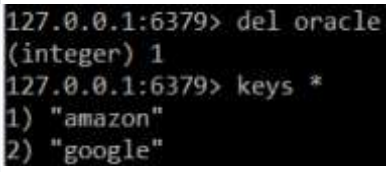
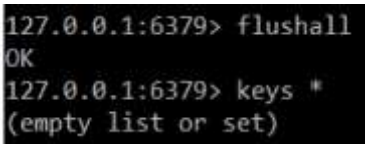
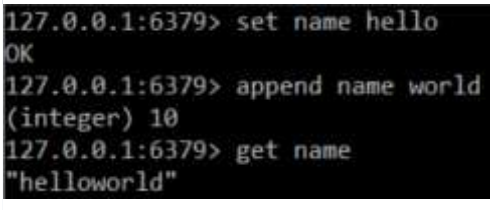
A visual representation of these structures is provided below:



1. String Commands

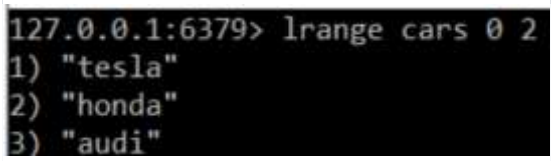
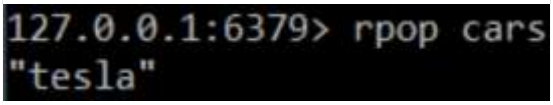
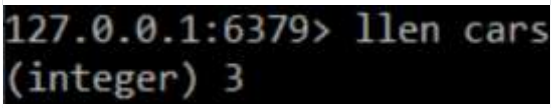
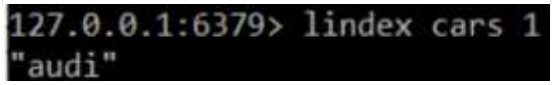
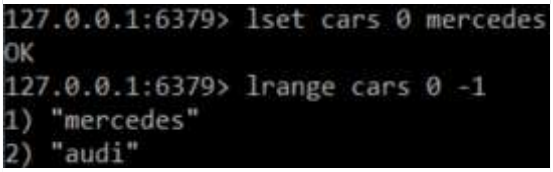
Strings are the simplest data type in Redis, used for storing text or binary data.

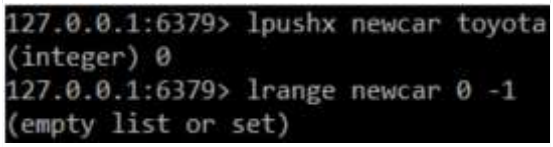
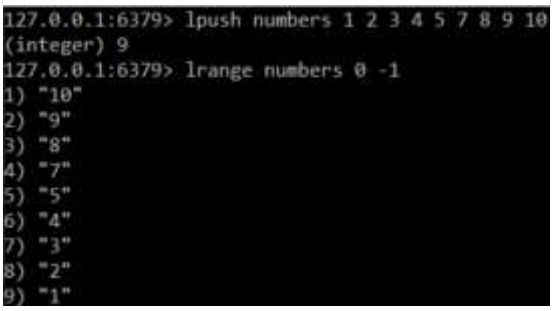
Command	Syntax	example	Description
SET	SET key value		Stores a record in Redis. If the key exists, it updates the value.
GET	GET key		Retrieves the value associated with the specified key.
SETEX	SETEX key seconds value		Sets a key with a value and an expiration time in seconds.
PSETEX	PSETEX key millisecond value		Sets a key with a value and an expiration time in milliseconds.
SETNX	SETNX key value		Sets the value of a key only if the key does not already exist.
STRLEN	STRLEN key		Returns the length of the value stored at the specified key.
MSET	MSET key1 value1 key2 value2 ...		Sets multiple key-value pairs in a single command.
MGET	MGET key1 key2 ...		Retrieves the values of multiple keys in a single command.
KEYS	KEYS *		Returns all the keys saved in the database.

Command	Syntax	example	Description
INCR	INCR key		Increments the integer value of a key by 1.
INCRBY	INCRBY key count		Increments the integer value of a key by a specified number.
INCRBYFLOAT	INCRBYFLOAT key floatValue		Increments the value of a key by a floating-point number.
DECR	DECR key		Decrements the integer value of a key by 1.
DECRBY	DECRBY key count		Decrements the integer value of a key by a specified number.
DEL	DEL key		Deletes a particular key or multiple keys from Redis.
FLUSHALL	FLUSHALL		Flushes (deletes) all the keys present in Redis.
APPEND	APPEND key stringToBeAppended		Appends a string to the existing value of a key.

2. List Command

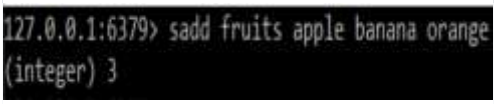
Lists are stored as linked lists, allowing insertion and removal from both the head (left) and tail (right).

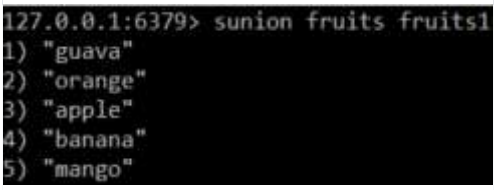
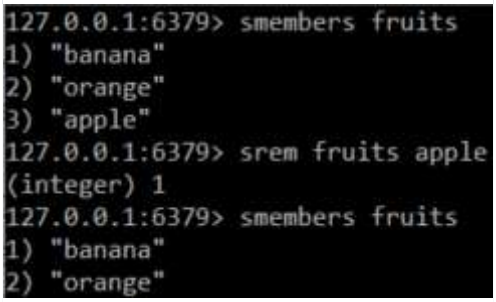
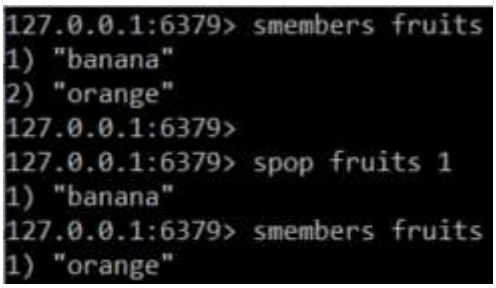
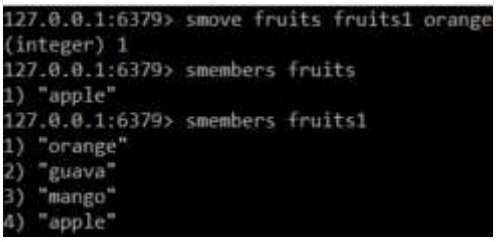
Command	Syntax	Example	Description
LPUSH	LPUSH key value...		Inserts one or more values at the head (left) of the list.
LRANGE	LRANGE key start end		Returns a range of elements from the list. Use -1 for the end index to see all elements.
LPOP	LPOP key		Removes and returns the element at the head (left) of the list.
RPUSH	RPUSH key value...		Inserts one or more values at the tail (right) of the list.
RPOP	RPOP key		Removes and returns the element at the tail (right) of the list.
LLEN	LLEN key		Returns the length of the list.
LINDEX	LINDEX key index		Finds the element at a particular index (starting from 0).
LSET	LSET key index value		Updates the value at a given index.

Command	Syntax	Example	Description
LPUSHX	LPUSHX key value		Adds an element to the head of the list only if the list exists.
LINSERT	LINSERT key BEFORE|AFTER pivot value		Inserts an element before or after a specified pivot element.

3. Set Commands

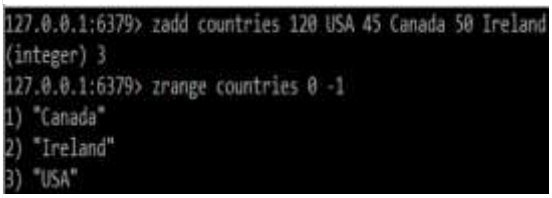
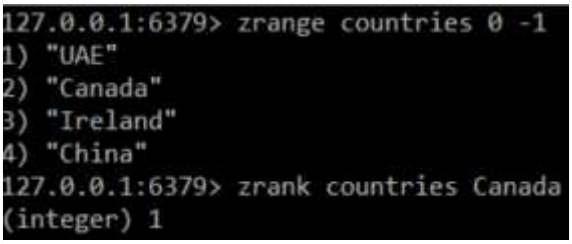
Sets are collections of unique, unordered elements. Redis uses a hash table internally for sets.

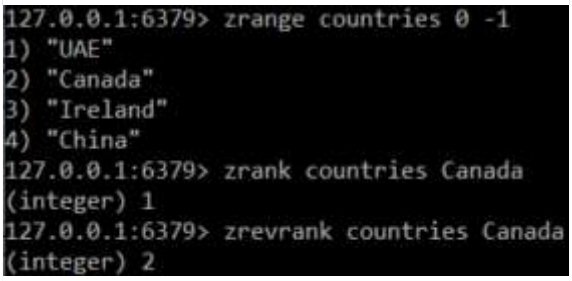
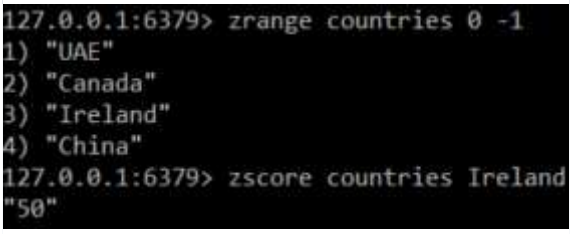
Command	Syntax	Example	Description
SADD	SADD key value		Adds an element to the set. Returns 0 if the element is already present.
SMEMBERS	SMEMBERS key		Returns all elements present in the set.
SISMEMBER	SISMEMBER key value		Checks if an element is present in the set (returns 1 if present, 0 otherwise).
SCARD	SCARD key		Returns the count of members present in the set.

Command	Syntax	Example	Description
SDIFF	SDIFF key1 key2		Returns the difference between two sets (elements in key1 but not in key2).
SUNION	SUNION key1 key2 ...		Returns the union of two or more sets.
SREM	SREM key value1 value2 ...		Removes one or more elements from the set.
SPOP	SPOP key count		Removes and returns one or more random values from the set.
SMOVE	SMOVE source dest member		Moves a value from a source set to a destination set.

4. Sorted Sets (ZSets) Commands

Sorted Sets are similar to Sets but every member is associated with a score, used to keep the set in order.

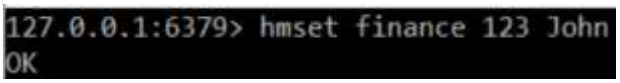
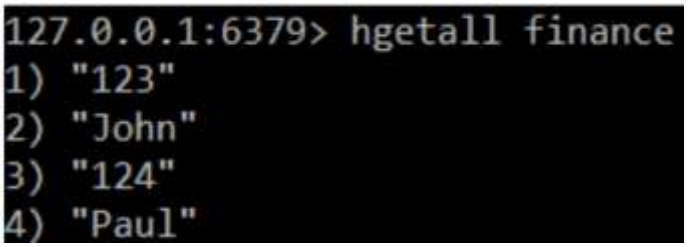
Command	Syntax	Example	Description
ZADD	ZADD key score value ...		Adds elements with scores to the sorted set. Elements are sorted in ascending order of score.
ZRANGE	ZRANGE key start end		Fetches elements in a range. Use -1 for the end to fetch all elements.
ZRANGEBYSCORE	ZRANGEBYSCORE key start end		Fetches elements within a particular range of scores.
ZCARD	ZCARD key		Returns the total number of elements in the sorted set.
ZCOUNT	ZCOUNT key min max		Returns the number of elements within a specific range of scores.
ZREM	ZREM key value		Removes a member from the sorted set.
ZRANK	ZRANK key member		Finds the index (rank) of an element (0 is the lowest score).

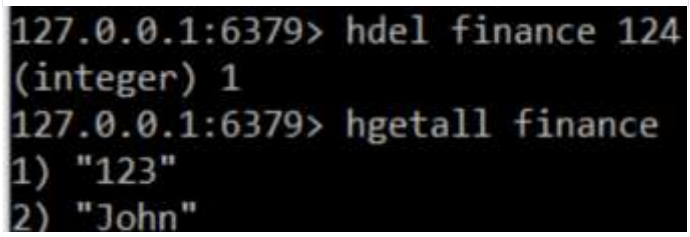
Command	Syntax	Example	Description
ZREVRANK	ZREVRANK key member		Finds the rank of an element from the reverse (0 is the highest score).
ZSCORE	ZSCORE key member		Retrieves the score associated with a specific member.

---

5. Hashes Commands

Hashes are maps between string fields and string values, ideal for representing objects.

Command	Syntax	Example	Description
HMSET	HMSET key field value	 	Stores a field-value pair in a hash. Can also store multiple pairs.
HGETALL	HGETALL key		Retrieves all field-value pairs for a given key.
HGET	HGET key field		Retrieves the value of a specific field within a hash.

Command	Syntax	Example	Description
HEXISTS	HEXISTS key field		Checks if a particular field exists in the hash (returns 1 if exists, 0 otherwise).
HDEL	HDEL key field		Deletes a specific field from the hash.
HSETNX	HSETNX key field value		Inserts a field only if it is not already present in the hash.

Using Redis as a Cache and Session Store

1. Redis as a Cache

Caching involves storing copies of frequently accessed data in memory to reduce the load on primary databases and improve application response times.

Common Caching Patterns

Pattern	Description	Best For
Cache-Aside (Lazy Loading)	The application first checks the cache. If data is missing (cache miss), it fetches it from the database and updates the cache.	Read-heavy workloads where data doesn't change frequently.
Write-Through	Data is written to the cache and the database simultaneously.	Ensuring the cache is always up-to-date with the database.
Write-Behind (Write-Back)	Data is written to the cache first, and the database is updated asynchronously after a delay.	Write-heavy workloads where database performance is a bottleneck.

Best Practices for Caching

- **Set Expiration (TTL)**: Always use Time-To-Live (TTL) to ensure stale data is eventually removed.
- **Eviction Policies**: Configure maxmemory-policy (e.g., allkeys-lru) to handle what happens when the cache is full.
- **Cache Invalidation**: Have a clear strategy for updating or deleting cache entries when the underlying data changes.

2. Redis as a Session Store

A session store tracks user state (login status, shopping carts, preferences) across multiple requests in a web application.

Why Use Redis for Sessions?

- **Speed**: Session data is read and written on almost every request; Redis's in-memory speed ensures no perceptible lag.
- **Scalability**: In a distributed environment (multiple web servers), a centralized Redis store allows any server to access the user's session.
- **Persistence**: Unlike simple in-memory stores, Redis can persist session data to disk, preventing user logouts during server restarts.

Implementation Workflow

1. **Session Creation**: When a user logs in, a unique Session ID is generated.
2. **Data Storage**: Session data is stored in Redis as a **Hash** or **String**, with the Session ID as the key.
3. **Automatic Cleanup**: Use Redis's EXPIRE command to automatically delete sessions after a period of inactivity (e.g., 30 minutes).

3. Comparison: Cache vs. Session Store

Feature	Redis as a Cache	Redis as a Session Store
Data Source	Copy of data from a primary database.	The primary source for temporary user state.
Loss Impact	Performance degradation (must re-fetch from DB).	User inconvenience (forced logout, lost cart).

Feature	Redis as a Cache	Redis as a Session Store
Typical TTL	Minutes to Days.	15 Minutes to 2 Hours (Inactivity-based).
Data Structure	Often Strings or Hashes.	Primarily Hashes (for field-level updates).

Redis Pub/Sub Messaging Model

the concepts behind using Redis as a queue.

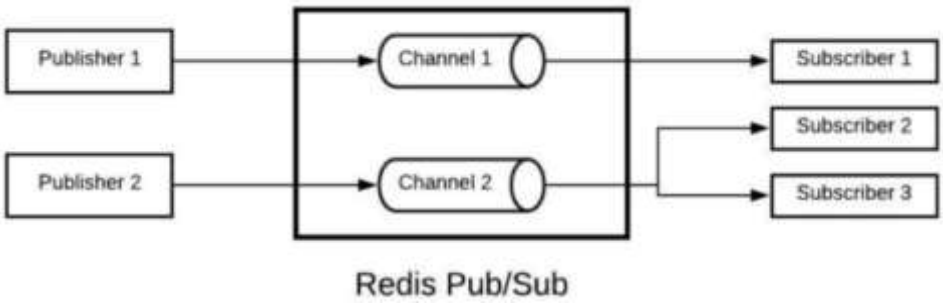
What is a message queue?

A message queue is a type of temporary storage for messages. The sender, also known as **publisher**, sends messages to the queue. The messages remain in the queue until a receiver also known as a **subscriber**, is ready to read the messages. The main use case for a message queue is when there are some larger jobs that need to be processed. The sender sends the jobs to a queue instead of directly sending it to the receiver. The receiver picks each job, one by one, and processes it.

Let's say we are booking movie tickets on a website. There are a few things that should be done immediately, like selecting the seats, payment, etc. But a few things, like sending tickets via email, may take time and are not required to be done immediately. These tasks that time and are not required to be done immediately can be added to a queue and can be processed one by one.

How Redis is used as a message queue?

As we discussed earlier, Redis can also be used as a message queue. The publisher can publish to only one channel, although a channel may have multiple subscribers. A subscriber can subscribe to one or more channels.



Working of a Publisher/Subscriber model in Redis

Now we will look at how the Redis can be used as a message queue.

Subscribing to a channel.

A user can subscribe to a channel using the SUBSCRIBE command. This command takes the name of the channel as an argument. The syntax of this command is:

SUBSCRIBE *channel*

```
127.0.0.1:6379> SUBSCRIBE mychannel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "mychannel"
3) (integer) 1
```

Publishing to a channel

A user can publish to a channel using the PUBLISH command. The syntax of this command is:

PUBLISH *channel message*

In the example below, the client is publishing a message to **mychannel**.

```
127.0.0.1:6379> PUBLISH mychannel hello
(integer) 1
```

The message is sent to the other client.

```
127.0.0.1:6379> SUBSCRIBE mychannel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "mychannel"
3) (integer) 1
1) "message"
2) "mychannel"
3) "hello"
```

A channel with multiple subscribers

It is possible for different clients to subscribe to the same channel. Whenever a publisher publishes this channel, all the receivers receive the message.

In the example below, there are two clients subscribed to **mychannel**. When a publisher sends a message to this channel, both of the subscribers receive the update.

```
E:\softwares\redis\Redis-7001\redis-cli.exe
127.0.0.1:6379> PUBLISH mychannel hello
(integer) 1
127.0.0.1:6379> PUBLISH mychannel helloworld
(integer) 2
127.0.0.1:6379>
```

```
E:\softwares\redis\Redis-7001\redis-cli.exe
127.0.0.1:6379> SUBSCRIBE mychannel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "mychannel"
3) (integer) 1
1) "message"
2) "mychannel"
3) "hello"
1) "message"
2) "mychannel"
3) "helloworld"
```

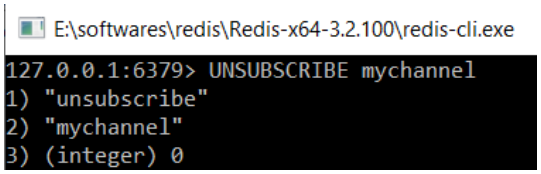
```
E:\softwares\redis\Redis-7001\redis-cli.exe
127.0.0.1:6379> SUBSCRIBE mychannel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "mychannel"
3) (integer) 1
1) "message"
2) "mychannel"
3) "helloworld"
```

Unsubscribing from a channel in Redis

A client can easily unsubscribe from a channel using the UNSUBSCRIBE command. The syntax of this command is

UNSUBSCRIBE *channel*

In the example below, the client is unsubscribing from **mychannel**.



Subscribing to a group of channels

If a client wants to subscribe to all the channels starting with a particular character or string, they can use the PSUBSCRIBE command. The syntax of this command is:

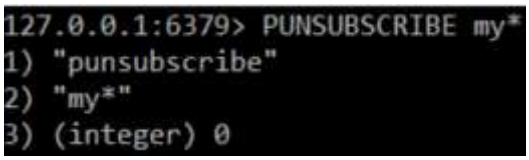
PSUBSCRIBE *string**

In the example below, a client is subscribing to a channel that starts with **my**.

Unsubscribing from a group of channels

A client can unsubscribe to a group of channels using PUNSUBSCRIBE command.

In the example below, a client is unsubscribing to all the channels starting with **my**.



Persistence Mechanisms: RDB and AOF

Redis: Persistence

Redis is an in-memory database, which means the data is stored in memory(RAM). In the case of a server crash, all the data stored in the memory is lost. To prevent this, Redis has some mechanism in place to back up the data on the disk. In case of a failure, the data can be loaded from the disk to the memory when the server reboots.

Redis provides two persistence options: RDB and AOF

RDB persistence(snapshotting)

In RDB(Redis Database) persistence, snapshots of the data are stored in a disk in a dump.rdb file. The interval for snapshotting is defined in the configuration file. The default configuration present in the redis.conf file is:

save 900 1

save 300 10

save 60 10000

In the example above, the behavior will be to save data as follows:

- After 900 sec (15 min), if at least 1 key changed
- After 300 sec (5 min), if at least 10 keys changed
- After 60 sec, if at least 10000 keys changed

If the load is too high and a lot of data is being stored, Redis will take a backup every minute. If the load is medium, the backup will be taken after every five minutes and so on.

The durability of data depends upon how frequently the data is being dumped to the disk. If data is dumped after every five minutes, and the server crashes after minutes since the last dump then all the data stored within the last four minutes are lost. So, the duration of backup should be decided carefully.

A user can also save the data in an rdb file using the **SAVE** command. This command blocks the Redis server, so it should be avoided. Instead, the **BGSAVE** command, which creates a child process for snapshotting, should be used.

Snapshotting can be disabled by deleting all save directives in the redis.conf file and then restarting the Redis server.

Configuration options in RDB

There are a few directives available in the **redis.conf** file that help us configure how data is persisted. We will discuss some of them here.

1. **rdbcompression:** If the value of this directive is true, Redis uses LZF compression. By default, this value is true, but, if you want to save some CPU cycles in the child process, then you can make it false.
2. **rdbchecksum:** If the value of this directive is yes, a CRC64 checksum is placed at the end of the file. This makes the format more resistant to corruption, but there is a performance hit to pay (around 10%) when saving and loading RDB files, so you can disable it for maximum performance. RDB files created with checksum disabled have a checksum of zero, which will tell the loading code to skip the check.
3. **stop-writes-on-bgsave-error:** If this option is true, Redis will stop accepting writes if the latest background save failed. If the background saving process starts working again, Redis will automatically allow writes again. The default value is true. It is advised not to change it to false because if there are some issues in saving the data on the disk that the user must know about it.
4. **dbfilename:** This option is used to specify the .rdb filename. The default value is dump.rdb

Performance impact of snapshotting

Redis creates a child process to write data to the disk. If some data is changed while the child process is writing the data to the disk, the child process needs to make a copy of that data as well. This can be time-consuming, and if the data set is large, it may force Redis to stop serving clients for anywhere between a millisecond to a second. The AOF method, which we will discuss next, is far better in terms of performance.

Append-only file (AOF) persistence

When AOF is enabled, every write operation received by the server is logged into a file. When the Redis is restarted, all the commands in the AOF file are run again, and the entire dataset is created.

After a while, this file can get really big, since it contains the entire history of every key. To tackle this problem, Redis rewrites that file every once in a while to keep it as small as possible. So, instead of storing the entire history of a key, it starts with the latest state of that key.

Let's say we enter a key **count** in Redis with the value **1**. It is changed multiple times through the set command. The AOF file will look like this:

set count 1

set count 4

set count 12

set count 56

When Redis rewrites this file, it will simply write the following command in the file, as all other commands are not needed.

set count 56

We can configure how often this rewrite happens based on the current file size and the allowed increase percentage since the size of the latest rewrite.

Even though Redis sends the new command to be appended to the file, the OS usually keeps these commands in a buffer and flushes that buffer every X number of seconds. This means that if there is a power failure while the commands are in the buffer, you may lose some data. Redis forces a buffer flush every second so that you don't lose anything. We can also configure Redis to force flush on every single command, but that makes Redis's responses per command very slow.

Configuration options in AOF

1. appendonly: This directive is used to enable the AOF option in Redis. By default, the value is false. To enable AOF, change this valueto true and restart the server.
2. appendfilename: This specifies the AOF filename. The default value is appendonly.aof
3. appendfsync: There is an fsync() call in the operating system, that tells the OS to flush data from the buffer to the disk. The fsync may happen every second or every ten seconds. It is upto the OS. Redis can control this time through the appendfsync directive. It has three options:

- **no**: Let the OS decide when to flush.

- **always**: Flush on every write. This makes Redis slow, but it is the safest option.
- **everysec**- Flush every second.

4. no-appendfsync-on-rewrite: When the AOF fsync policy is set to always or everysec, and a background saving process is performing a lot of I/O against the disk, Redis may block the fsync() call for too long. If this option is true, fsync() will not be called in the main process while a BGSAVE or BGREWRITEAOF is in progress. This means, that while another child is saving, the durability of Redis is the same as appendfsync none.
5. auto-aof-rewrite-percentage: This directive specifies when an automatic rewrite of an AOF file should happen. If the value of this property is 20, Redis will automatically rewrite the log file by implicitly executing the BGREWRITEAOF command when the AOF size grows by 20 percent. The default value is 100.
6. auto-aof-rewrite-min-size: This is the minimum size for AOF to be rewritten. This prevents AOF rewrites until the specified minimum size is reached, even if the specified auto-aof-rewrite-percentage value is exceeded. The default value is 67,108,864 bytes

Which persistence strategy should be preferred?

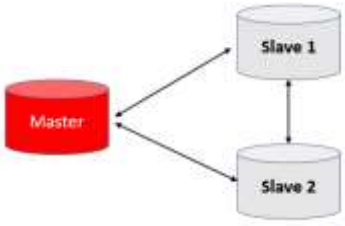
Redis uses snapshotting as the default strategy. If you are fine with losing a few seconds of data, then this strategy should be used. If you need complete durability and can't afford to lose any data, then AOF should be used. You can also use both strategies but Redis will use AOF to populate data, as it is the most accurate one.

Replication and High Availability

What is data replication?

When data is stored on a server and there is a server crash, there is a chance that the data will be lost. To avoid this, the technique of data replication is used. The data is stored on two or more servers instead of a single server. That way, if a server goes down, data is not lost since it is available on the other server. Also, the application does not crash because the other server can cater to the user requests. Another benefit of data replication is that it can reduce the load on the servers because the user requests can be load balanced to the servers instead of sending it to a single server.

Replication in Redis



Master-slave architecture for data replication

Redis follows a *master-slave* approach for replication. One of the servers is a master, and the other servers are called slaves. The slaves are connected to the master. All the writes happen to the master and then the master, sends these changes to the slaves. The reads can then be handled by the slaves. Through this process, the load is distributed.

If a slave gets disconnected from the master, it automatically reconnects and tries to be the exact copy of the master. There are two approaches that a slave takes to get in sync with the master.

- **Partial Synchronization**: The slave tries to get only the data that it missed while it was disconnected. The slave sends the master information about the point to which it has the data. We will discuss this in more detail later.
- **Full Synchronization**: If partial synchronization is not possible, then full synchronization takes place. The master sends all the data to an rdb file. This rdb file is sent to the slave. The slave then saves it to its disk. After that is done, the slave loads the data from the rdb file to its memory. There is an option of *disk-less replication* as well. The child process can directly send the rdb to replicas over the wire, without using the disk as intermediate storage.

Synchronization of master and salves

1. The master is non-blocking and continues to handle queries when one or more replicas perform the initial synchronization or a partial re-synchronization.
2. The slave is also non-blocking. When a slave is synchronizing the data, it will also serve the clients from the old data if it is configured to do so.
3. When a slave gets disconnected from the master, it may or may not serve the clients. It depends on the slave-serve-stale-data configuration. If this configuration is set to yes, then the slave will serve out-of-date data to the client. If it is false, then the slave will reply with a **SYNC with master in progress** error to all the commands except INFO and SLAVEOF.
4. The slave sends a ping to the master after regular intervals. This tells the master if a slave is still connected or not. By default, a slave sends the ping every ten seconds, but this can be configured using the repl-ping-slave-period property in the redis.conf file.
5. We can configure how much bandwidth is used in syncing the slave by using the repl-disable-tcp-nodelay configuration. If this value is **yes**, Redis will use a smaller number of TCP packets and less bandwidth to send data to slaves. This will delay the appearance of data on the slave side. If **no** is selected, then the delay will be reduced, but more bandwidth will be used for replication.
6. When a slave disconnects from a master, the data being transmitted is saved in a buffer. When a slave reconnects, it gets the data from this buffer. If a slave is disconnected for too long, this buffer may not have all the data that the slave missed, so full sync is required. The larger the size of the buffer, the higher the chances are of a partial sync. The default size of the buffer is 1 MB, but this can be configured using repl-backlog-size. The buffer is allocated only when the first slave is connected to the master.

7. If all the slaves are disconnected from the master, then the buffer is not required anymore. It is deleted after a certain period of time, which can be defined through the repl-backlog-ttl property. The default value is 3,600 seconds.
8. Each slave has a priority that is used to decide which slave will be promoted to master in case the master crashes. If there are three slaves with priorities 20, 30, and 50 then the slave with priority 20 will be promoted to master. The default priority of a server is 100. This can be configured using the slave-priority configuration. If this field is set to 0, then it means that a slave cannot be promoted to master.

Performance impact of replication in Redis

Since Redis stores data in-memory, it needs to store the data on the disk from time to time in order to protect the data in case of a failure. Storing the data on the disk is a costly operation so it is possible that, due to latency concerns, persistence is disabled on the master server. However, turning the persistence off is not advised unless it is absolutely necessary. If disabling persistence is necessary, the master should be configured to avoid restarting automatically after a reboot.

If persistence is disabled and the server restarts automatically, it may lead to data loss. If this happens, the slaves will try to sync with the master, and their data will also be lost.

How replication is done in Redis?

The replication process in Redis is asynchronous. The master sends the data to be replicated to the slave and does not check if it was actually saved in the slave server. The slave server asynchronously acknowledges the amount of data received periodically from the master. This gives the master an idea of what commands the slaves have processed.

We discussed earlier that if a slave is disconnected and reconnected, it syncs only the data that it missed. The question that arises here how the master knows what data the slave missed. The **replication ID** and **offset** are used to figure this out. Each master server has a **replication ID** that is also inherited by a slave. The master also has an offset that increments for every byte of replication stream that is produced to be sent to replicas. When a slave is reconnected, it sends its offset to the server.

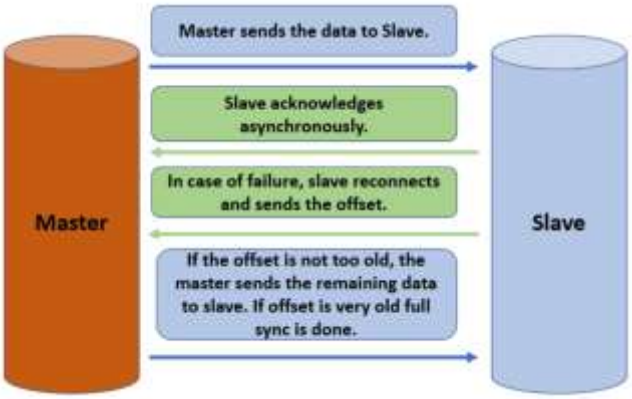
Let's suppose the master has a replication ID of **123gh3** and an offset as **12,000**. The slave has the replication ID of **123gh3** and the offset of **10,000**. When the slave connects with the master, it will send its offset of 10,000 to the master. The master will then know which bytes the slave is missing and it will resend those bytes to the slave.

There can be a situation where the offset provided by the slave is too old, and the master does not have that stream in the buffer. In that case, partial sync cannot be done and full sync is required.

Replication mechanism in Redis

How full sync is performed?

Full sync is done through the rdb file. The master creates an rdb file and sends it to the slave. The slave then saves it to the disk and loads the data into the memory. It is possible, however, that some writes are happening on the master after the master has created the rdb file. The master will store all these write operations on a buffer, the commands from the buffer will be sent to the slave after it is done loading the rdb file. Through this process, the slave will have the exact copy of the master.



Full sync mechanism in Redis

Configuring Replication in Redis

Let's discuss how to configure replication in Redis.

In the previous lesson, we discussed the concept of replication in Redis. Now it is time to see how replication can be configured in Redis. There are two ways to configure a Redis server as a slave.

1. Provide the slaveof property in the redis.conf file. This property takes two parameters, i.e., the host and the port of the master. An example of how this can be written in the conf file is shown below.

slaveof 127.0.0.1 6379

When this Redis instance is started, it will automatically connect to the master server.

2. We can use the REPLICAOF command to convert a Redis instance into a slave. Here are a few important points about this command:

- **REPLICAOF hostname port** will make the server a replica of another server listening at the specified hostname and port.
- If a server is already a slave and **REPLICAOF hostname port** is run on this server with different hosts, it will become the slave of a new master. It will delete the data of the old master and replicate the data of the new master.
- **REPLICAOF NO ONE** will convert a slave into a master. If a master is done, then this command can be used to convert its slave to the master. Later when the old master is up, it can be configured to become a slave.

Read-only replicas

By default, replicas are read-only. However, this behavior can be controlled using the replica-read-only property in the redis.conf file. If a replica is not read-only, it will accept write commands, but as soon as it syncs with master, the data written only on the replica will be lost. This raises the question of why we would need to write data to a replica if it is going to be deleted after syncing with the master. There are certain use cases in which this may be required. We may compute some data using the keys that are frequently used. This can be stored on the writable replica. This type of data is called *ephemeral data*.

Authenticating a replica with the master

In the Redis Security, we discussed that we can set a password for the master, and in order for a client to connect to the master, they must know the password. Similarly, a replica(slave) must also be aware of the master password to connect to it. There are two ways through which we can provide the master password to a slave.

1. If the slave is already running, the following command can be used to authorize it with the server.

config set masterauth <*password*>

2. We can also provide the master password in the redis.conf file of slave using the masterauth property, as shown below.

masterauth <*password*>

Minimum slaves configuration

We can configure the master to only accept writes if a certain number of slaves are connected to it. This does not ensure that the replica will actually receive all the writes. There are still some chances of data loss, but the chances are reduced.

A slave usually sends a ping to the master every second but it may get delayed. The time that has passed from when the last ping was sent is called the **lag**, e.g., if a master received a ping 5 seconds ago, the lag is five seconds. We can configure a master to stop accepting writes if there are less than N

slaves connected, having a lag less than or equal to M seconds. This is done using the two parameters shown below:

min-slaves-to-write <number of replicas>

min-slaves-max-lag <number of seconds>

If min-slaves-to-write is set to 3 and min-slaves-max-lag is set to 10, it means that at least three slaves must have pinged the master within the last ten seconds.

Setting one or the other to 0 disables the feature. By default, min-slaves-to-write is set to 0 (feature disabled) and min-slaves-max-lag is set to 10.

High Availability (HA) with Redis Sentinel

While basic replication provides data redundancy, it does not handle **automatic failover**. If the master fails, a replica must be manually promoted. **Redis Sentinel** automates this process.

Core Functions of Sentinel

- **Monitoring**: Constantly checks if master and replica instances are working correctly.
- **Notification**: Alerts administrators or other systems via an API when an instance fails.
- **Automatic Failover**: If a master fails, Sentinel promotes a replica to master, reconfigures other replicas to follow the new master, and informs applications of the new address.
- **Configuration Provider**: Acts as a service discovery source for clients to find the current master.

Sentinel as a Distributed System

Sentinel is designed to run as a cluster of multiple processes (typically at least three). This prevents the HA system itself from being a single point of failure.

- **Quorum**: The number of Sentinels that must agree a master is down to trigger a failover.
- **Leader Election**: Once a failure is detected, the Sentinels vote to elect a "leader" Sentinel to execute the failover.

3. High Availability with Redis Cluster

For larger deployments, **Redis Cluster** provides both high availability and **horizontal scaling (sharding)**.

Feature	Redis Sentinel	Redis Cluster
Primary Goal	High Availability	HA + Horizontal Scaling (Sharding)

Feature	Redis Sentinel	Redis Cluster
Data Distribution	Every node has a full copy of the data.	Data is partitioned across multiple master nodes.
Failover	Managed by external Sentinel processes.	Managed internally by the cluster nodes themselves.
Scaling	Scales reads (via replicas).	Scales both reads and writes (via sharding).

4. Summary: Replication vs. High Availability

Concept	Definition	Key Component
Replication	The mechanism of copying data from a master to replicas.	PSYNC, RDB Snapshots, Command Stream
High Availability	The ability of the system to remain operational after a failure.	Redis Sentinel or Redis Cluster

Redis Cluster Introduction

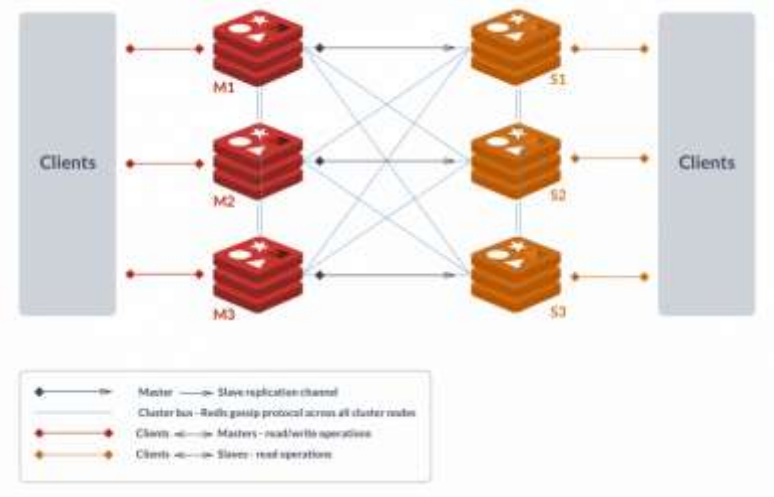
we discussed many concepts regarding replication and partitioning. Although the concepts seemed simple, it is difficult to handle sharding if the nodes are constantly being added or removed. The Redis cluster is a way to do all of this automatically.

After over four years of development, Redis Cluster was released with the 3.0.0 release of Redis on April 1, 2015. “According to the founder, it is:

“…basically a **data sharding strategy**, with the ability to reshard keys from one node to another while the cluster is running, together with a failover procedure that makes sure the system is able to survive certain kinds of failures.””

The two major advantages of Redis cluster are:

- The ability to automatically split the dataset among multiple nodes.
- The ability to continue operations when a subset of the nodes are experiencing failures or are unable to communicate with the rest of the cluster.



Working of Redis cluster. Source : https://redislabs.com/redis-features/redis-cluster

Redis cluster working

In Redis cluster, each node has two TCP connections open:

1. The normal port which is used for client communication, such as 6379. This connection is available to all clients as well as those cluster nodes that need to use client connection for key migration.
2. The cluster bus port, which is used by other cluster nodes for failure detection, configuration updates, and fail-over authorization. This channel uses less bandwidth by utilizing a binary protocol for data exchange. This port is always 10000 + the normal port. So if the normal port is 6379, then the cluster bus port will be 16379.

Hash slots

Redis cluster uses **hash slots** to shard the keys. A Redis cluster has 16,383 slots. These slots are distributed among the servers. So, if there are 3 servers in a cluster, then:

- Server A contains hash slots from 0 to 5,500.
- Server B contains hash slots from 5,501 to 11,000.
- Server C contains hash slots from 11,001 to 16,383.

It is not necessary for the slots to be equally distributed among the servers.

Whenever a new server is added or deleted, these slots are redistributed. Moving hash slots from a node to another does not require you to stop operations. Thus, adding and removing nodes, or changing the percentage of hash slots held by nodes does not require any downtime.

When a key is inserted in Redis, the hash slot is found using the CRC-16 hash function to convert the key into an integer. Then modulo 16383 of that integer is calculated to get the **hash slot** for this key.

HASH_SLOT = CRC16(key) mod 16383

Hash tags

The Redis cluster allows multiple key operations, if the keys are on the same node and have the same hash slot. The user can force multiple keys to be part of the same hash slot by using **hash tags**.
In **hash tags**, a sub-string within {} is added between the key. The hash is then calculated using this sub-string. Thus, all the keys return the same hash. Let's say we are storing a few keys in Redis, and we want them to end up in the same node. We will insert a random substring(let's say abcd) in between our keys, e.g., 23464{abcd}2344, relat{abcd}ed, pre34{abcd}pared. When these keys are inserted, the hash slot will be calculated using the string, **abcd**. Thus, all the keys will have the same hash slot.

Fault tolerance in Redis cluster

The sharding of data won't help in the case of a server failure. If a server goes down all the keys stored on that server will be lost. To handle this kind of situation, replication is required. Redis Cluster uses a master-slave model, where every hash slot has anywhere from one (the master itself) to N replicas (N-1 additional slave nodes). To be considered healthy, a cluster should have atleast three master nodes and one replica for each master. So, a cluster should have at-least 6 nodes. M1, M2, and M3 are the master nodes, and S1, S2, and S3 are the replicated nodes.
Redis cluster writes data to the slave node asynchronously. This means that when a key is inserted or updated, the master node sends this information to the slave node but does not wait for acknowledgement. If the master node crashes, it is possible that the slave does not contain all the keys that a client has entered. This problem can be solved by storing the data in a slave node synchronously, but doing this will slow down the server. Thus, Redis cluster makes a trade-off between performance and consistency.

Redis cluster configuration

By default Redis runs as a single-instance server. If we want to run Redis in cluster mode then we need to provide a new set of directives in the redis.conf file. These directives are defined below.

1. cluster-enabled: This property should be **yes** if we need to start the Redis instance in cluster mode. If this property is **no**, then this instance cannot be used in a Redis cluster.
2. cluster-config-file: This directive sets the path for the configuration file that stores the changes happening to the cluster. This file is created by Redis and this directive should not be changed by the user. It stores things like the number of nodes in the cluster, their state etc. When a Redis cluster is started, this file is read.
3. cluster-node-timeout: This property defines the maximum amount of time in milliseconds, that a node can be unavailable without being considered failed. If a master node is not reachable for more than the specified amount of time, then a slave node is promoted to master. If a slave node is not reachable then it stops accepting requests from clients.
4. cluster-slave-validity-factor: If a master is disconnected from the cluster, then a slave tries to fail-over a master. If the value of this property is 0, then a slave will always try to fail-over a

master, regardless of the amount of time the link between the master and the slave remains disconnected.

However, it is possible that a master is disconnected from a slave for so long that the slave is no longer the right candidate to become a master because it has very old data. This time is calculated using this property.

max time = cluster-node-timeout * cluster-slave-validity-factor

So, if **cluster-node-timeout** is 2 and **cluster-slave-validity-factor** is 10, then the maximum timeout is 20 milliseconds. If a slave is disconnected from a master for more than 20 milliseconds, it is considered unsuitable and cannot be promoted to master.

5. cluster-migration-barrier: This directive defines the minimum number of slaves that a master will remain connected with before another slave migrates to a master that is no longer covered by any slave. Let's say we have two masters, A and B. A has two slaves, called A1 and A2. B has one slave, called B1. If the master B goes down, then B1 is promoted to master. Now master B does not have any slaves. If **cluster-migration-barrier** is set to 2, then B1 will not be able to borrow a slave from A because 2 is the minimum requirement.
6. cluster-require-full-coverage: If a master server fails and it does not have any slaves, then the keys that were stored on this server are lost. If this property is set to **yes**, as it is by default, then the entire cluster will become unavailable. If the option is set to no, then the cluster will still serve the queries, but the keys that are lost will return an error.

Creating and Testing a Redis Cluster

Let's learn how to create a Redis cluster.

There are two ways to create a Redis cluster, the easy way and the hard way. The easy way is to create a cluster using the **create-cluster** command. The hard way is to configure the Redis servers to create a cluster manually. In this lesson, we will look at the hard way.

Some of the commands used to create a cluster were introduced in Redis 5. Since the highest version supported in Windows is 3.2, it will not be possible to create a cluster in Windows using the method described in this lesson.

Creating Redis instances

The first step in creating a cluster is to create multiple Redis instances that will be added to the cluster. We will create six instances. Three will be masters, and three will be slaves.

We have created six folders and installed a Redis instance in each folder using the below commands.

$ wget http://*download.redis.io/redis-stable.tar.gz*

$ tar xzf redis-stable.tar.gz

$ cd redis-stable

$ make

Creating a Redis instance in each of the six folders

We have selected the folder names based on the ports that will be used for each server.

We will need to provide some configuration in the **redis.conf** file to start the server in cluster mode. Provide the following configuration in the redis.conf file of each instance.

port 7001

cluster-enabled yes

cluster-config-file nodes.conf

cluster-node-timeout 10000

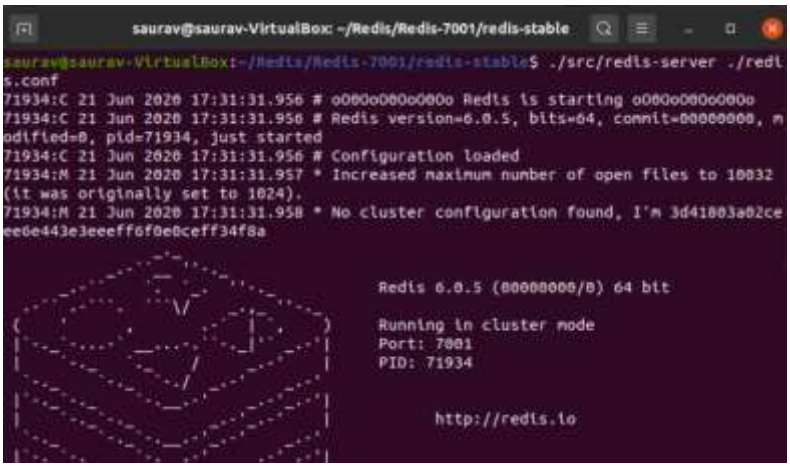
appendonly yes

The port will be different for each instance.

Once this configuration is done, open six terminals and start each instance using the following command:

./src/redis-server ./redis.conf

The screenshot of the instance running at port 7001.



Instance running at port 7001

When all six instances are running, our next step is to create a cluster.

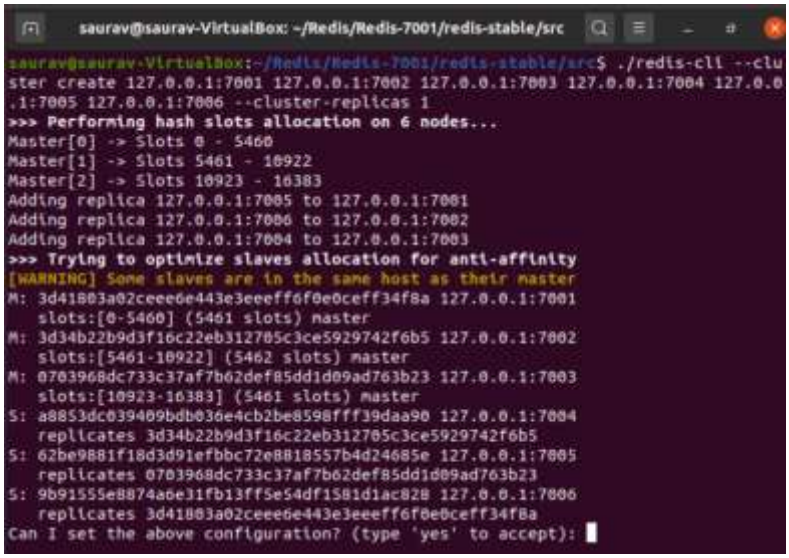
Creating a cluster

To create a cluster, run the following command in a new terminal:

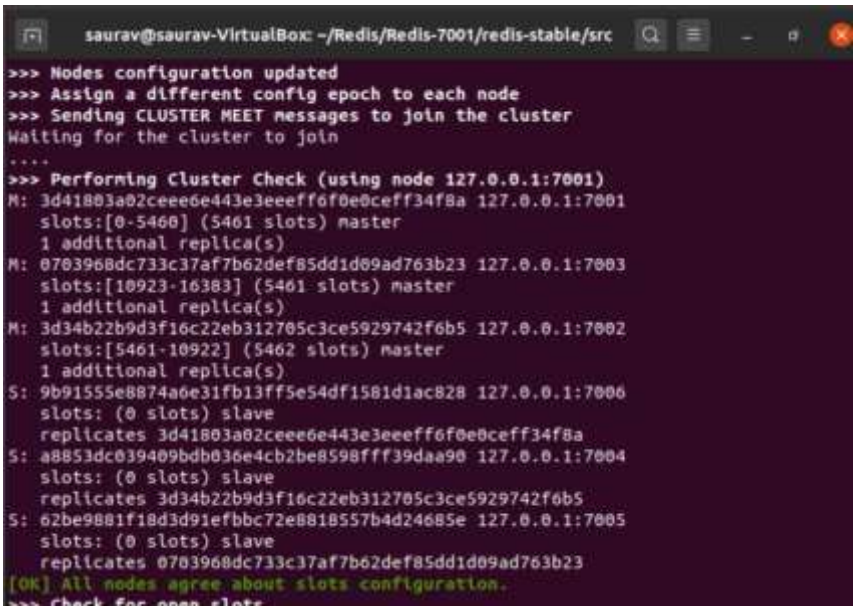
./redis-cli --cluster create 127.0.0.1:7001 127.0.0.1:7002 127.0.0.1:7003 127.0.0.1:7004 127.0.0.1:7005 127.0.0.1:7006 --cluster-replicas 1

The **–cluster-replicas 1** option means that each master should have one slave. Since we have six instances, three masters and three slaves will be created.

In the below image, you can see that 3 masters are created and the slots are divided among the masters. Three slaves are also created.



Creation of three masters



Creation of three slaves

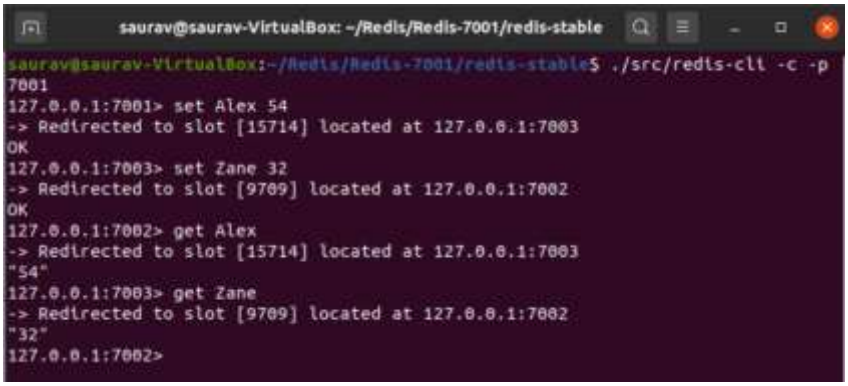
Now that our cluster is created, the next step is to connect to the cluster and execute some commands.

Connecting to a cluster

To connect to a cluster, we will need to connect to one of the nodes. The command to connect to the instance running on port 7001 is shown below. This command is used to indicate that the client should be using the cluster mode.

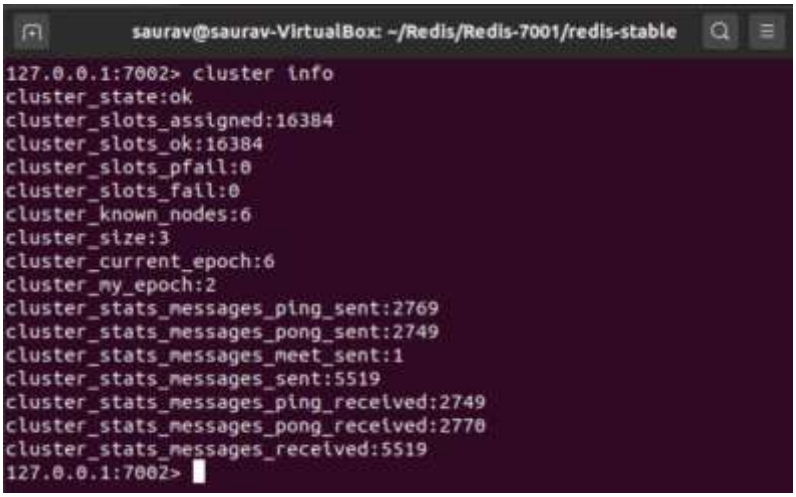
./src/redis-cli -c -p 7001

In the image below, you can see that the keys are getting stored in different instances based on the slot calculated.



Keys stored in different instances based on the slots calculated

The **CLUSTER INFO** command can be used to check the cluster status as shown below. It shows that the cluster state is ok, and all the slots are assigned.



Checking the cluster status using the cluster info command

Adding and Removing Instances from a Cluster

Let’s discuss how to add and remove nodes from a cluster.

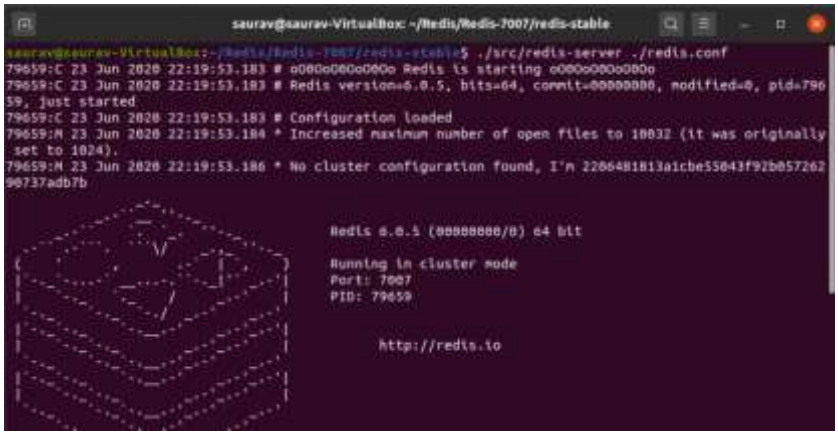
In the previous lesson, we created a cluster with six nodes. There may be certain situations where we need to add or remove instances from the cluster. Let’s look at the commands used to add and remove nodes from a cluster.

Adding nodes to a cluster

When a new node is added to a cluster, it is considered a master. It does not have any hash slots assigned, so it cannot store any key. It is up to us to determine which hash slots we need to move to the newly added master. We can move the hash slot from one master to another and then migrate the keys stored on that hash slot. The step-by-step guide to adding a new node to the cluster is shown below.

Creating a new instance

In our cluster, we have six nodes running on ports 7001, 7002, 7003, 7004, 7005 and 7006. We will create a new instance that will be run on port 7007. The process of creating this instance is the same as the process shown in the previous lesson.



Creating a new instance

Introducing the new instance to the cluster

The **CLUSTER MEET** command is used to introduce a new instance to the cluster. Run the following command in a new terminal to inform the cluster that a new node is available.

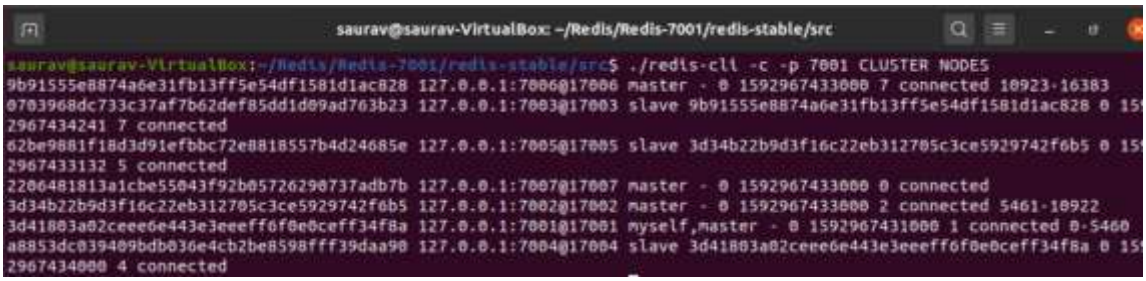
redis-cli -c -p 7001 CLUSTER MEET 127.0.0.1 7007

We have introduced the new instance to the instance running on port 7001. This is enough as we don't need to introduce the new instance to each node in the cluster.

Introducing the new instance to the cluster

Reshard the hash slots

We will move the hash slot, **9709**, which is currently on the node running at port 7002. To move the hash slot, we will first need to find out the source and destination node id. This can be done using the **CLUSTER NODES** command.



Finding the hash slot using the CLUSTER NODES command

The steps to migrate a hash slot from one node to another are listed below:

a) Import a hash slot

The receiving node will run a command to inform the cluster that it wishes to import a hash slot. The command for this is:

CLUSTER SETSLOT <hash-slot> IMPORTING <source-id>

Here, **hash-slot** is the slot that needs to be moved, and **source-id** is the node id of the node on which this hash slot is currently residing. In our case, we need to import a hash slot from the node running on port 7002. So, we will be running the following command from the node that is going to receive the hash slot.

CLUSTER SETSLOT 9709 IMPORTING 3d34b22b9d3f16c22eb312705c3ce5929742f6b5

Importing a hash slot

b) Migrate the hash slot

Once the receiving node has run the import command, the source node will run the migrate command. The command for migrating a hash slot is:

CLUSTER SETSLOT <hash-slot> MIGRATING <destination-id>:

Here, **destination-id** is the node id of the destination server.

Since we are moving the slot to a node running on port 7007, we will run the following command:

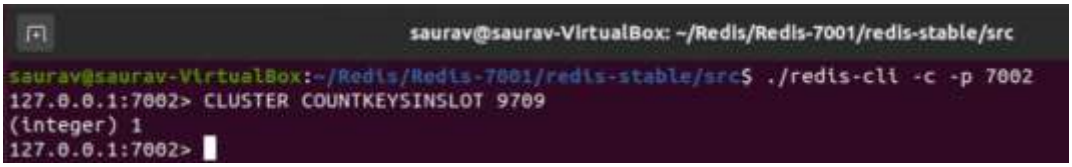
CLUSTER SETSLOT 9707 MIGRATING 2206481813a1cbe55043f92b05726290737adb7b

Migrating the hash slot

c) Migrating the keys

We have migrated the hash slot, 9709, to the new node. Now it's time to move the keys that were stored on this slot to the new node. The following command will tell us how many keys are in a particular slot.

CLUSTER COUNTKEYSINSLOT 9707



Finding the total keys in a particular slot

To migrate the key, we should know the key name as well. This can be found using the following command:

CLUSTER GETKEYSINSLOT <slot> <amount>

Here, amount is the number of key names we want to get.

Finding the key name

Once we know the key name, we can run the following command to migrate the key.

MIGRATE <host> <port> <key> <db> <timeout>

Migrating the key

Informing all the nodes about hash slot migration

All the nodes in the cluster should be informed of the new location of the hash slot. This is required so that other nodes can look for the key at the correct location. The command for this operation is:

CLUSTER SETSLOT <hash-slot> NODE <owner-id>:

Here, **owner-id** is the node id of the instance where the hash slot has been moved.

This command should only be run for master nodes.

$ redis-cli -c -p 7001 CLUSTER SETSLOT 9709 NODE

2206481813a1cbe55043f92b05726290737adb7b

$ redis-cli -c -p 7002 CLUSTER SETSLOT 9709 NODE

2206481813a1cbe55043f92b05726290737adb7b

$ redis-cli -c -p 7003 CLUSTER SETSLOT 9709 NODE

2206481813a1cbe55043f92b05726290737adb7b

Removing nodes from a cluster

To remove a node, first, we need to migrate all its hash slots to different nodes. A node can only be removed from a cluster when there are no hash slots assigned to it.

After all the slots are migrated from a node, then the following command should be run on all the master nodes.

CLUSTER FORGET <node-id>

As soon as CLUSTER FORGET is executed, the node is added to a ban list. This ban list exists in order to avoid adding the node back to the cluster when nodes exchange messages. The expiration time for the ban list is 60 seconds, so the above command should be executed in all of the master servers within 60 seconds.

Comparison of Redis with Traditional Relational Databases

This table consolidates the key differences between Redis, a high-performance in-memory data store, and Traditional Relational Databases (RDBMS) like MySQL, PostgreSQL, and Oracle.

Feature	**Redis**	**Traditional RDBMS (e.g., MySQL)**
Primary Storage	**In-Memory (RAM)**	**Disk-Based** (with in-memory caching)

Feature	Redis	Traditional RDBMS (e.g., MySQL)
Data Model	**Key-Value** and **Data Structures** (Lists, Sets, Hashes, Sorted Sets)	**Relational** (Tables with rows and columns)
Query Language	**Command-Based** (e.g., GET, LPUSH, HGETALL)	**SQL (Structured Query Language)**
Schema	**Schema-less** (Flexible)	**Fixed Schema** (Requires predefined structure)
Performance (Latency)	**Extremely Low** (Sub-millisecond)	**Moderate** (Millisecond range, dependent on disk I/O)
ACID Compliance	**Partial** (Focus on Atomicity for single operations)	**Full** (Designed for Atomicity, Consistency, Isolation, Durability)
Data Relationships	**None** (No native support for Joins)	**Strong** (Native support for Joins and Foreign Keys)
Primary Use Cases	Caching, Session Management, Real-time Analytics, Leaderboards, Message Queues	Transactional Systems, Complex Reporting, Systems of Record
Scalability	**Horizontal** (Clustering/Sharding is straightforward)	**Vertical** (More powerful hardware) and **Complex Horizontal** (Sharding is complex)
Data Volume	Limited by available **RAM**	Limited by available **Disk Space**